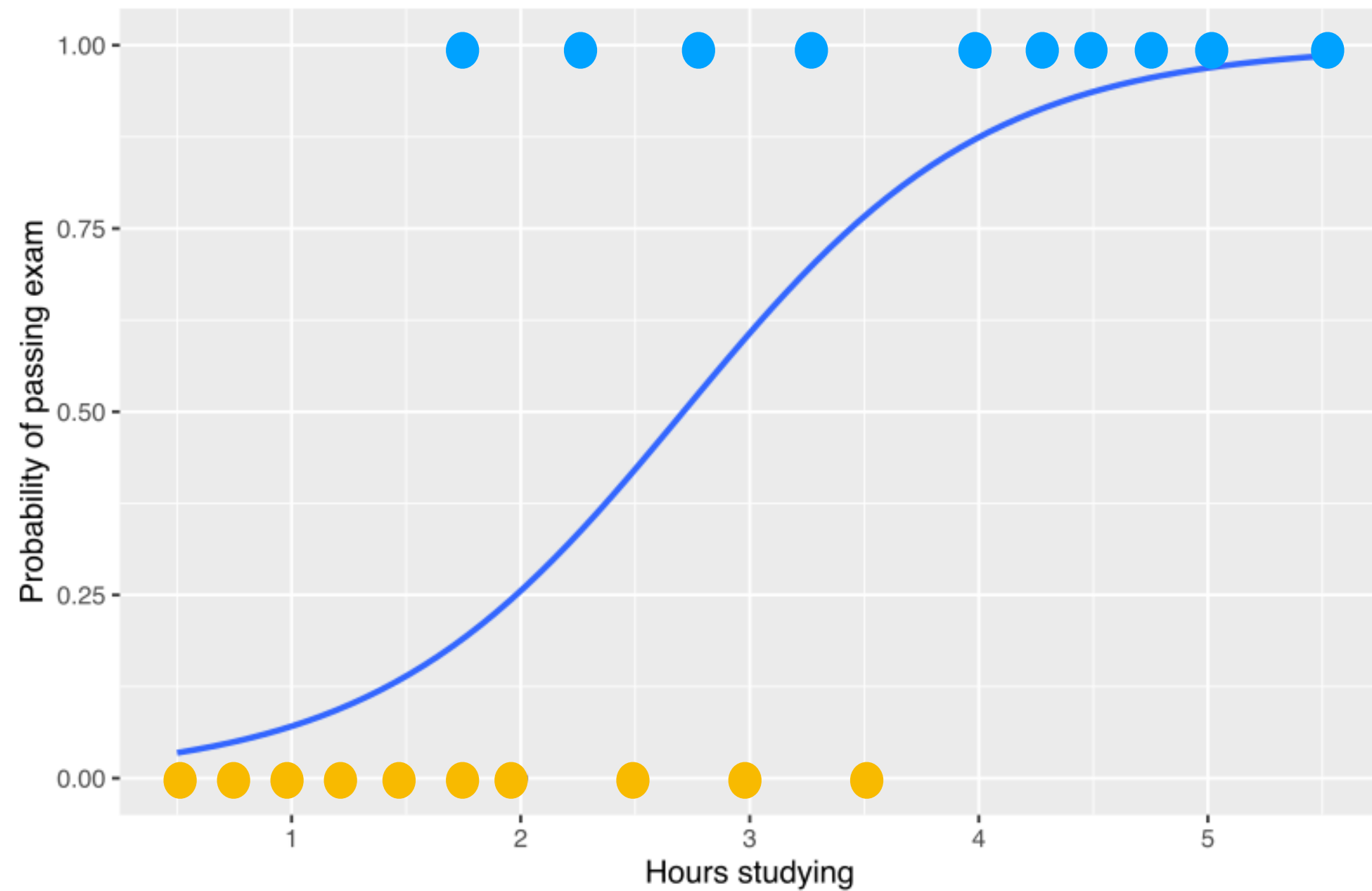


REVIEW OF BASICS OF LOGISTIC REGRESSION



Model the probability curve with the **logistic function**:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

Find w_0 , w_1 that minimizes **cross-entropy** loss:

$$-\sum_{i=1}^n (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

$$\text{where } p_i = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multiple logistic regression: What happens with multiple predictors?

Extends naturally. Logistic function:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots + w_m x_m)}}$$

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multi-class Classification

Ex: Movie classification into “drama”, “action”, “documentary”, etc.

How to handle it?

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multi-class Classification

Ex: Movie classification into “drama”, “action”, “documentary”, etc.

How to handle it?

Bad Idea: Assign $0, 1, 2, \dots$ to the classes, apply linear regression.

- What if classes can't be ordered naturally (data not ordinal)?
- **Ex:** drama:0, action:1, documentary=2, ...

drama < action < documentary means what?

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multi-class Classification

Ex: Movie classification into “drama”, “action”, “documentary”, etc.

How to handle it?

Bad Idea: Assign 0,1,2,... to the classes, apply linear regression.

- What if classes can't be ordered naturally (data not ordinal)?
- **Ex:** drama:0, action:1, documentary=2, ...

drama < action < documentary means what?

May work: For classes defined by *Likert scales*:

Completely Agree ↔ Mostly Agree ↔ Neutral ↔ Mostly Disagree ↔ Completely Disagree

Four stars ↔ Three stars ↔ Two stars ↔ One stars ↔ Zero stars

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multi-class Classification

Ex: Movie classification into “drama”, “action”, “documentary”, etc.

How to handle it?

Better Idea: Multiple *one-vs-rest* classifiers

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Multi-class Classification

Ex: Movie classification into “drama”, “action”, “documentary”, etc.

How to handle it?

Better Idea: Multiple *one-vs-rest* classifiers

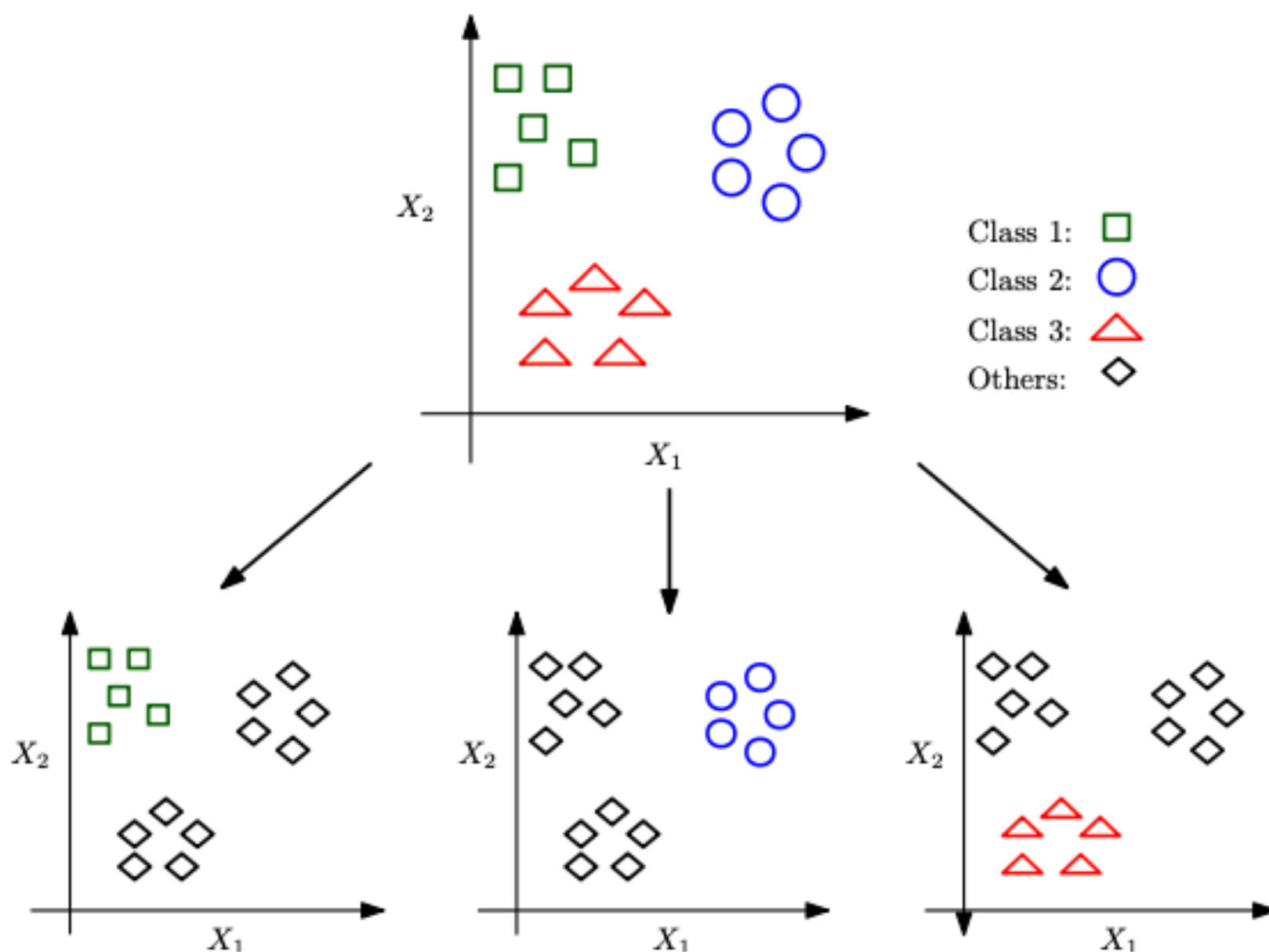
Train multiple logistic classifiers:

For each C_i : C_i against the rest

To identify the class of a new x :

Test against each classifier.

Pick class with highest probability.



Another Option: Multinomial regression (Probabilities now sum to 1)

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

`sklearn.linear_model.LogisticRegression`

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None) ¶
```

[\[source\]](#)



`multi_class='ovr'` implements one-vs-rest

`multi_class='multinomial'` implements multinomial

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Ex: Digits data from OCR

```
x, y = load_digits(return_X_y=True)
x.shape
```

```
(1797, 64)
```

```
x_train, x_test, y_train, y_test = \
    train_test_split(x, y, train_size=0.8, random_state=0)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)

model = LogisticRegression(multi_class='ovr', random_state=0)
model.fit(x_train, y_train)

x_test = scaler.transform(x_test)
y_pred = model.predict(x_test)
print(f"Train accuracy: {model.score(x_train, y_train)}")
print(f"Test accuracy: {model.score(x_test, y_test)}")
```

```
Train accuracy: 0.9930410577592206
Test accuracy: 0.9583333333333334
```

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Ex: Digits data from OCR

```
x, y = load_digits(return_X_y=True)
x.shape
```

```
(1797, 64)
```

```
x_train, x_test, y_train, y_test =\
    train_test_split(x, y, train_size=0.8, random_state=0)
```

```
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
```

```
model = LogisticRegression(multi_class='ovr', random_state=0)
model.fit(x_train, y_train)
```

```
x_test = scaler.transform(x_test)
y_pred = model.predict(x_test)
print(f"Train accuracy: {model.score(x_train, y_train)}")
print(f"Test accuracy: {model.score(x_test, y_test)}")
```

```
Train accuracy: 0.9930410577592206
Test accuracy: 0.9583333333333334
```

```
confusion_matrix(y_test, y_pred)
```

```
array([[27,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 33,  0,  0,  0,  0,  1,  0,  1,  0],
       [ 0,  0, 35,  1,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 29,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0,  0, 29,  0,  0,  1,  0,  0],
       [ 0,  1,  0,  0,  0, 39,  0,  0,  0,  0],
       [ 0,  1,  0,  0,  0,  0, 43,  0,  0,  0],
       [ 0,  0,  0,  0,  2,  0,  0, 37,  0,  0],
       [ 0,  3,  1,  0,  0,  0,  0,  0, 35,  0],
       [ 0,  0,  0,  1,  0,  1,  0,  1,  0, 38]])
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score
0	1.00	1.00	1.00
1	0.87	0.94	0.90
2	0.97	0.97	0.97
3	0.94	1.00	0.97
4	0.94	0.97	0.95
5	0.97	0.97	0.97
6	0.98	0.98	0.98
7	0.95	0.95	0.95
8	0.97	0.90	0.93
9	1.00	0.93	0.96

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Class Imbalance: Had already seen the issues related to it.

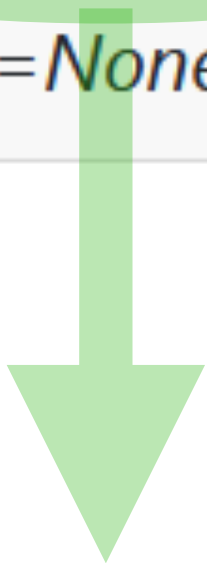
How to resolve?

- Discard members of the larger class to enforce balance.
- Replicate items from small class (with some random noise)
- Weigh the smaller class instances more in the loss function.

LOGISTIC REGRESSION

`sklearn.linear_model.LogisticRegression`

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1,  
class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False,  
n_jobs=None, l1_ratio=None) \[source\]
```



Can provide a dictionary of class & weight associations

OR

`class_weight='balanced'`: values of y used to automatically adjust weights inversely proportional to class frequencies in the input data

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Regularization

Large weights are possible (especially with linearly separable data)

To avoid them and reduce model complexity apply regularization:

- (Like linear regression) Make sure features are standardized

- Minimize

$$\textit{Cross Entropy} + \lambda \sum_{j=1}^m w_j^2$$

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

Regularization

Large weights are possible (especially with linearly separable data)

To avoid them and reduce model complexity apply regularization:

- (Like linear regression) Make sure features are standardized

- Minimize

$$\text{Cross Entropy} + \lambda \sum_{j=1}^m w_j^2$$

Can replace with: $\lambda \sum_{i=1}^m |w_j|$ for LASSO.

OTHER CONSIDERATIONS IN THE REGRESSION MODEL

`sklearn.linear_model.LogisticRegression`

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
```

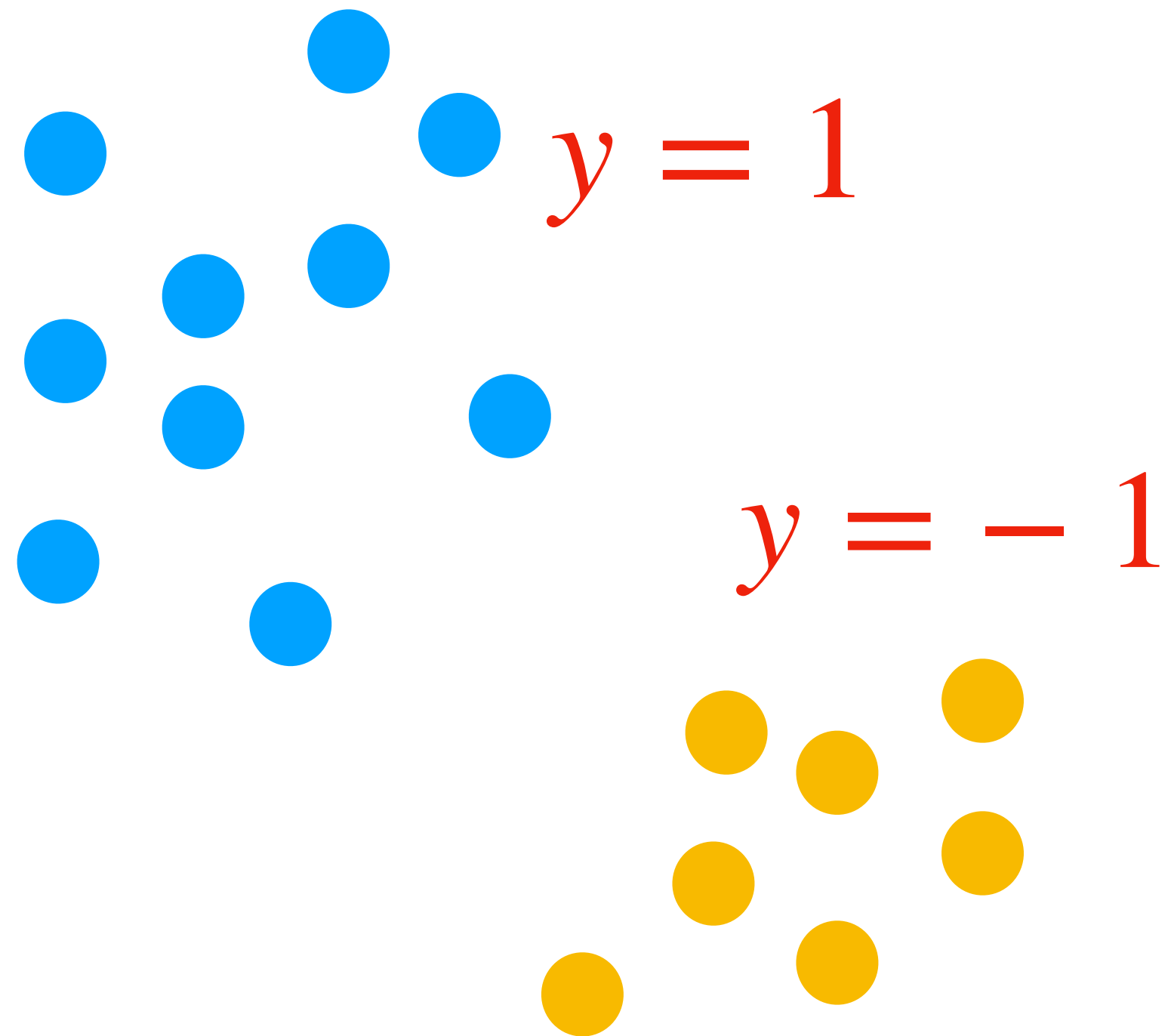
[\[source\]](#)

Make it '*l1*' for LASSO:
may have to assign specific solver.

$C = 1/\lambda$: For aggressive regularization set to a small value.

SUPPORT VECTOR MACHINES (SVM)

SVM Summary:



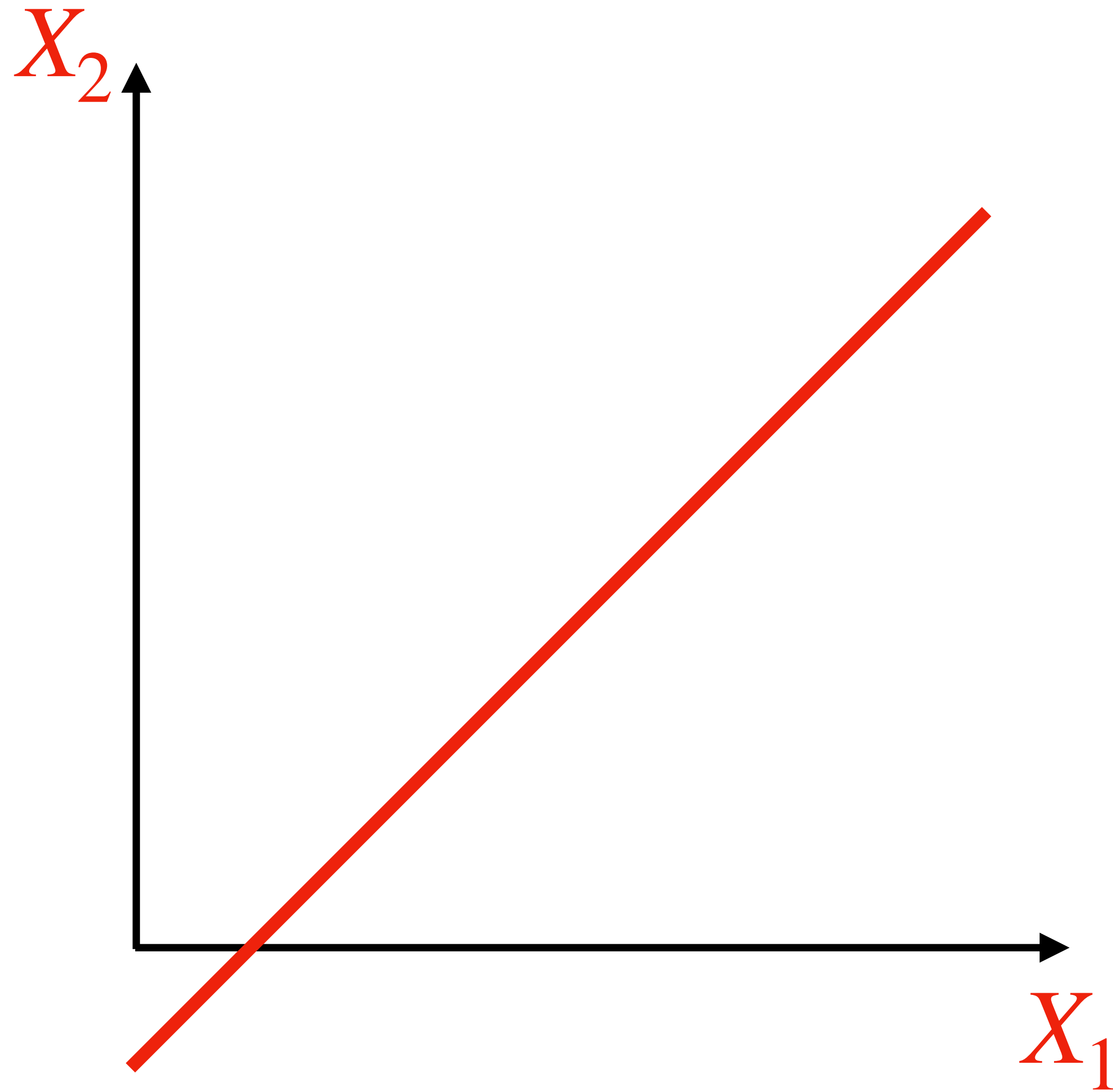
Let y denote the target variable.

For logistic regression we had labelled the classes 0,1.

For SVM let's label them with 1, -1 .

SUPPORT VECTOR MACHINES (SVM)

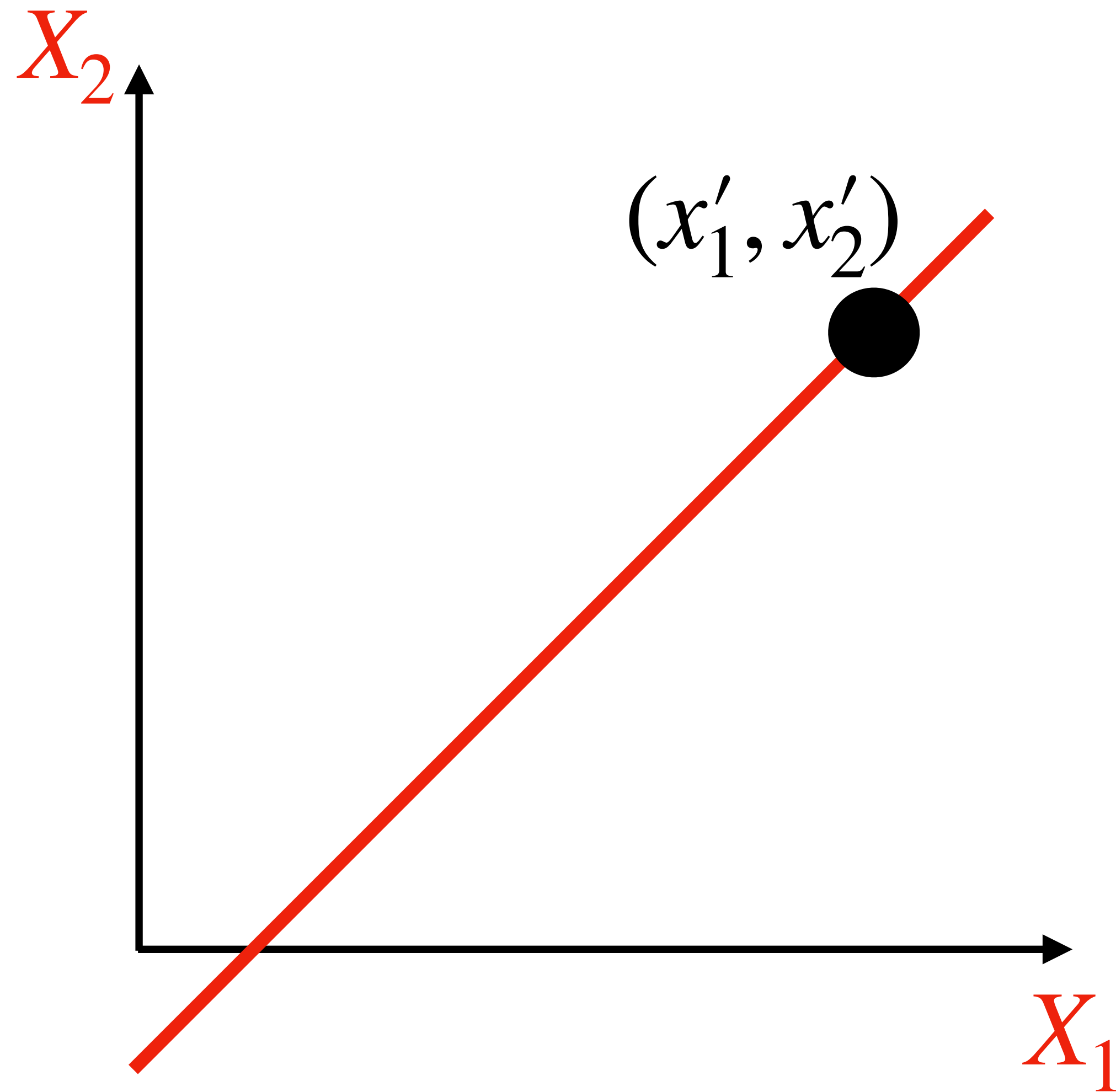
Hyperplane: Start with a hyperplane in 2D, which is a line.



$w_0 + w_1X_1 + w_2X_2 = 0$

SUPPORT VECTOR MACHINES (SVM)

Hyperplane: Start with a hyperplane in 2D, which is a line.



↓

$$w_0 + w_1X_1 + w_2X_2 = 0$$

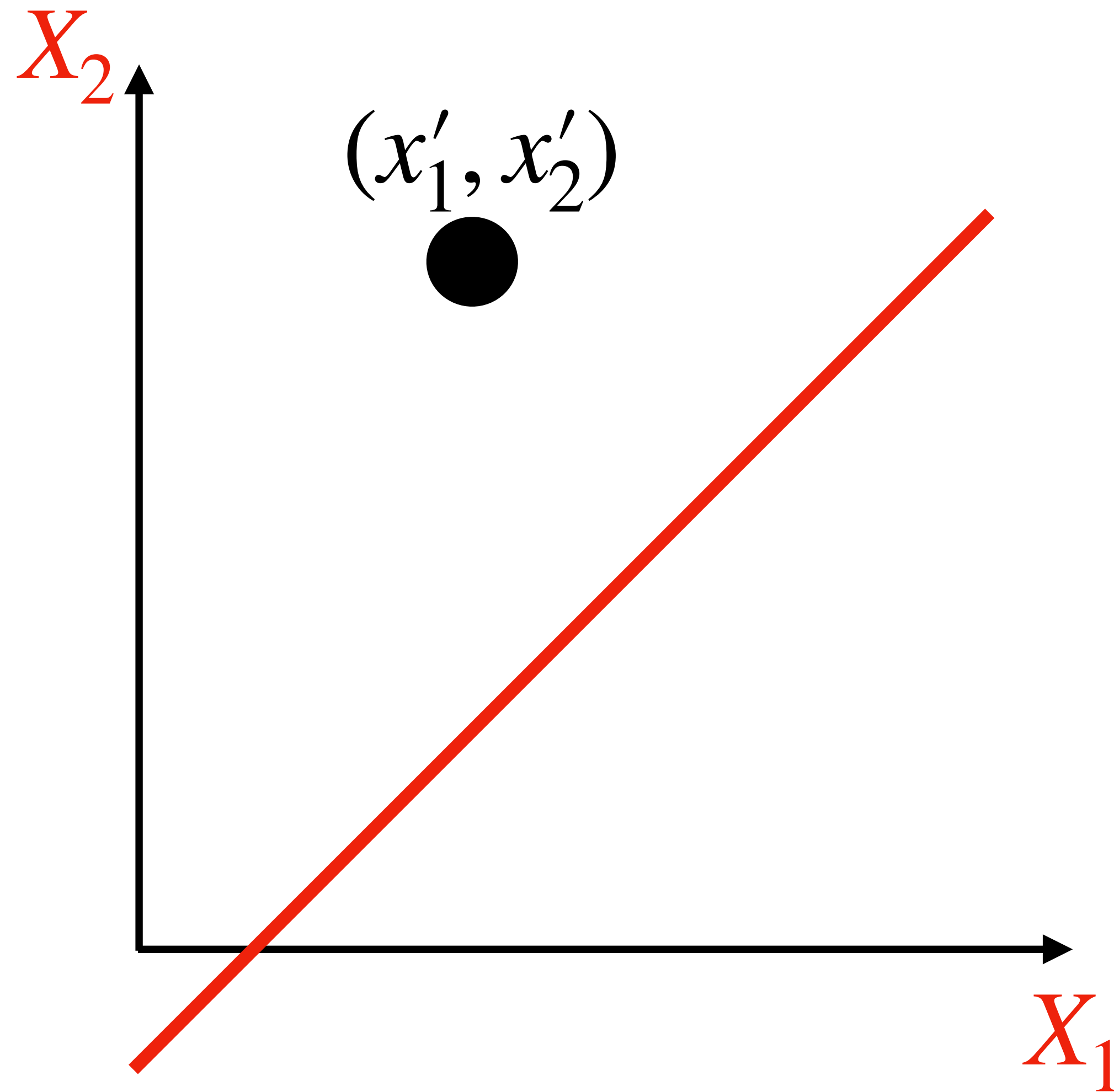
For a point (x'_1, x'_2) :

If $w_0 + w_1x'_1 + w_2x'_2 = 0$

then it is **on** the line.

SUPPORT VECTOR MACHINES (SVM)

Hyperplane: Start with a hyperplane in 2D, which is a line.



↓

$$w_0 + w_1X_1 + w_2X_2 = 0$$

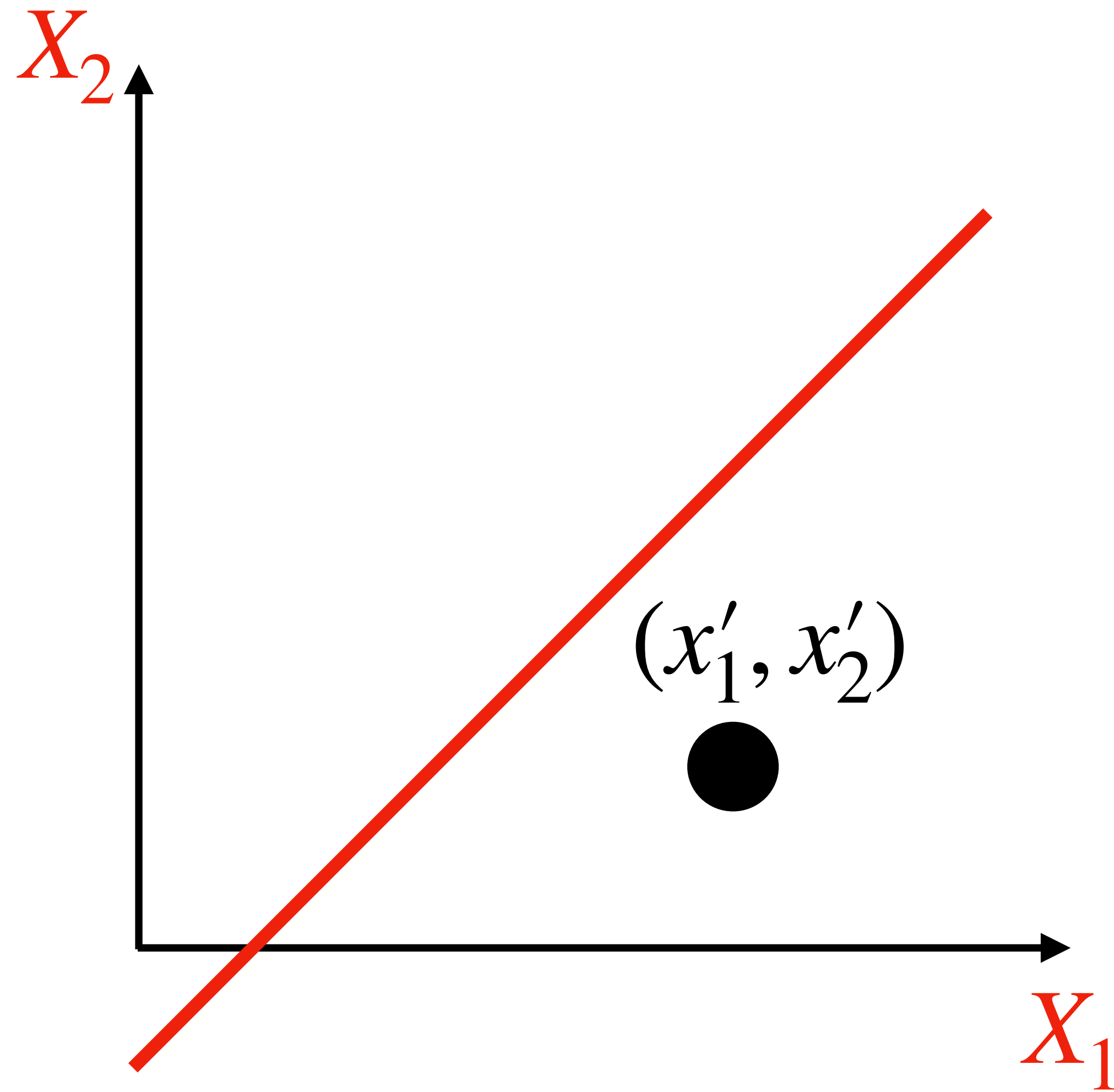
For a point (x'_1, x'_2) :

If $w_0 + w_1x'_1 + w_2x'_2 > 0$

then it is **above** the line.

SUPPORT VECTOR MACHINES (SVM)

Hyperplane: Start with a hyperplane in 2D, which is a line.



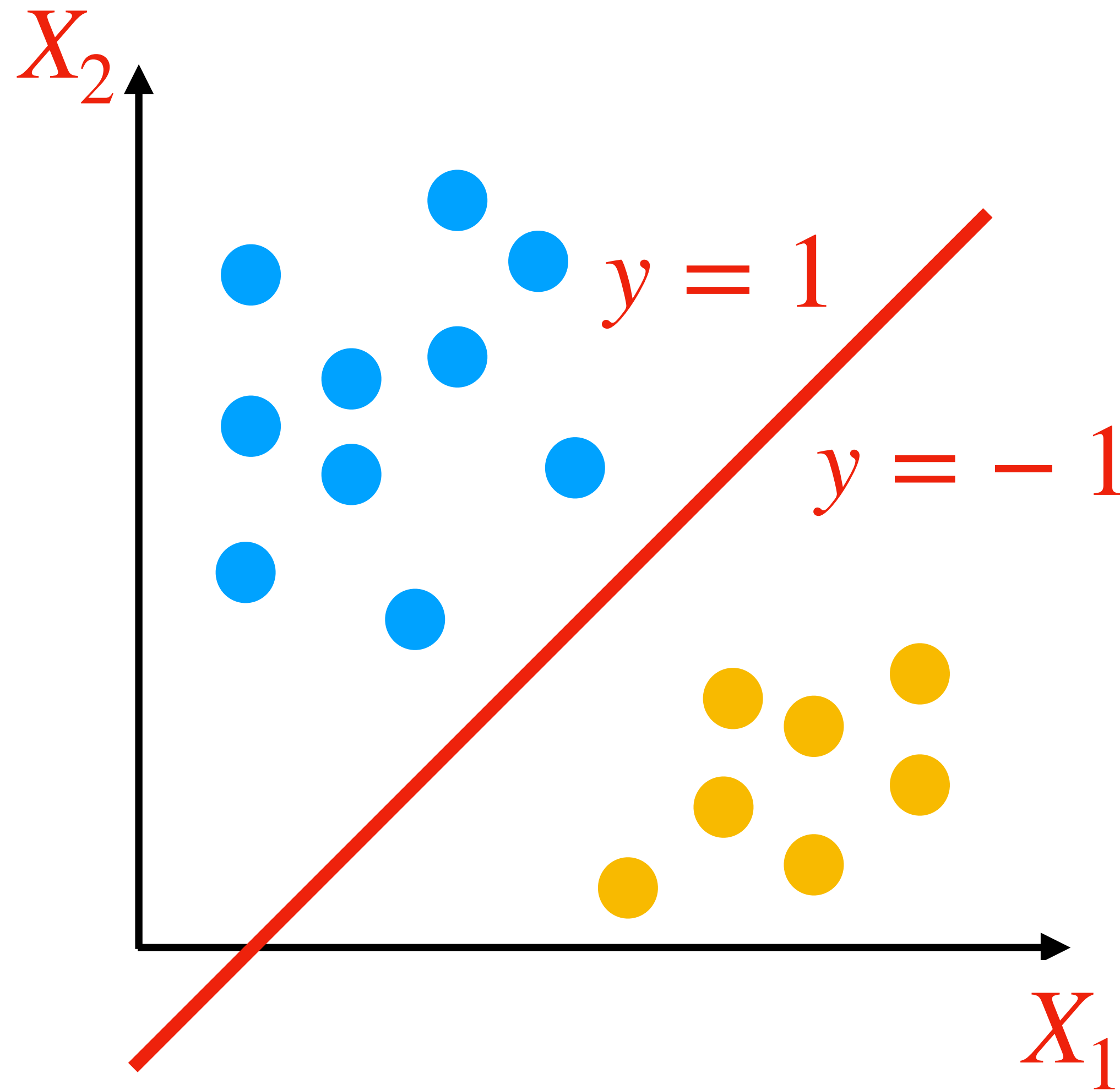
$$w_0 + w_1X_1 + w_2X_2 = 0$$

For a point (x'_1, x'_2) :

If $w_0 + w_1x'_1 + w_2x'_2 < 0$
then it is **below** the line.

SUPPORT VECTOR MACHINES (SVM)

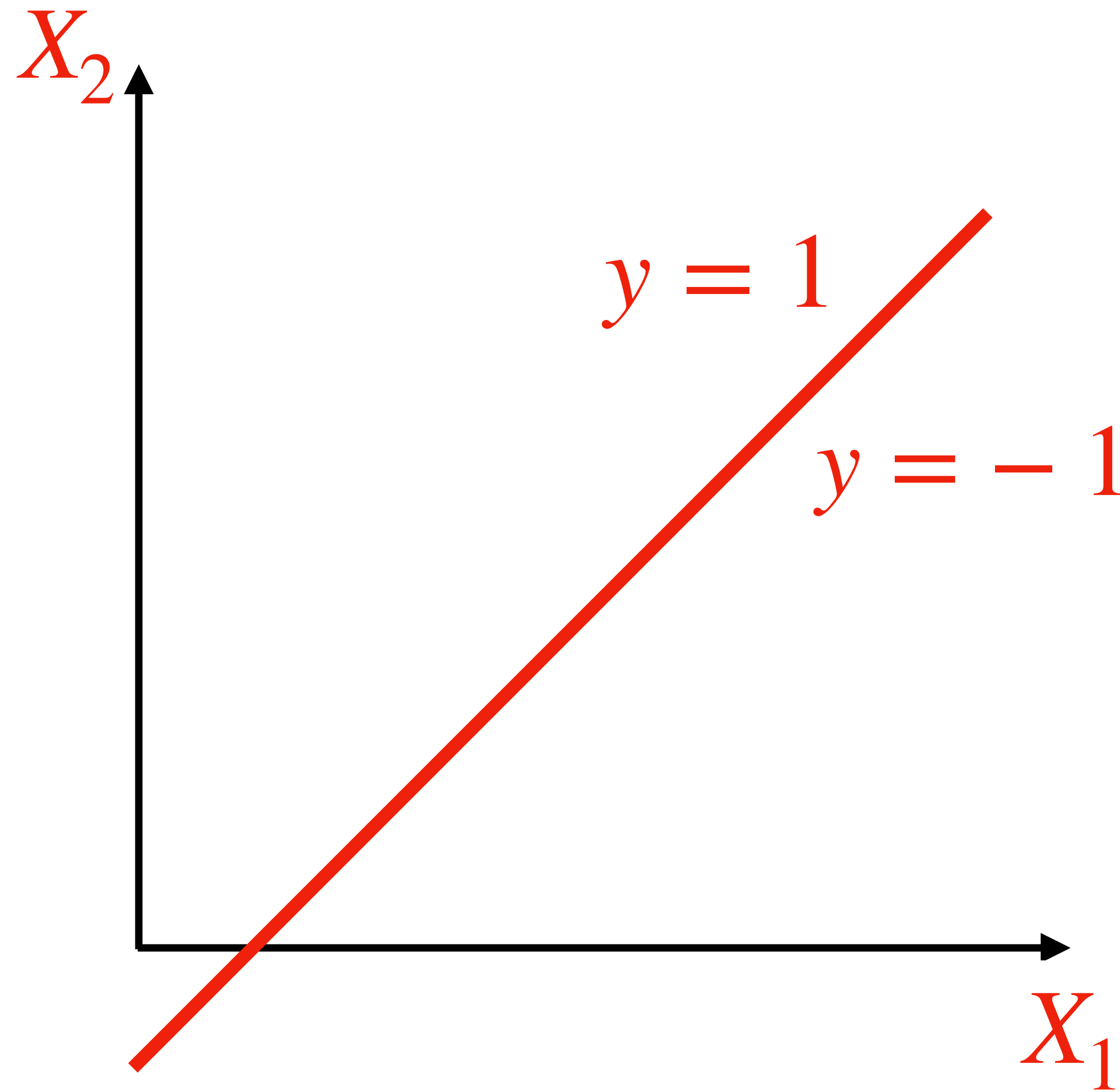
We want to find the hyperplane that
'separates' the classes 'well'.



SUPPORT VECTOR MACHINES (SVM)

We want to find the hyperplane that
'separates' the classes 'well'.

Meaning of 'separates classes':



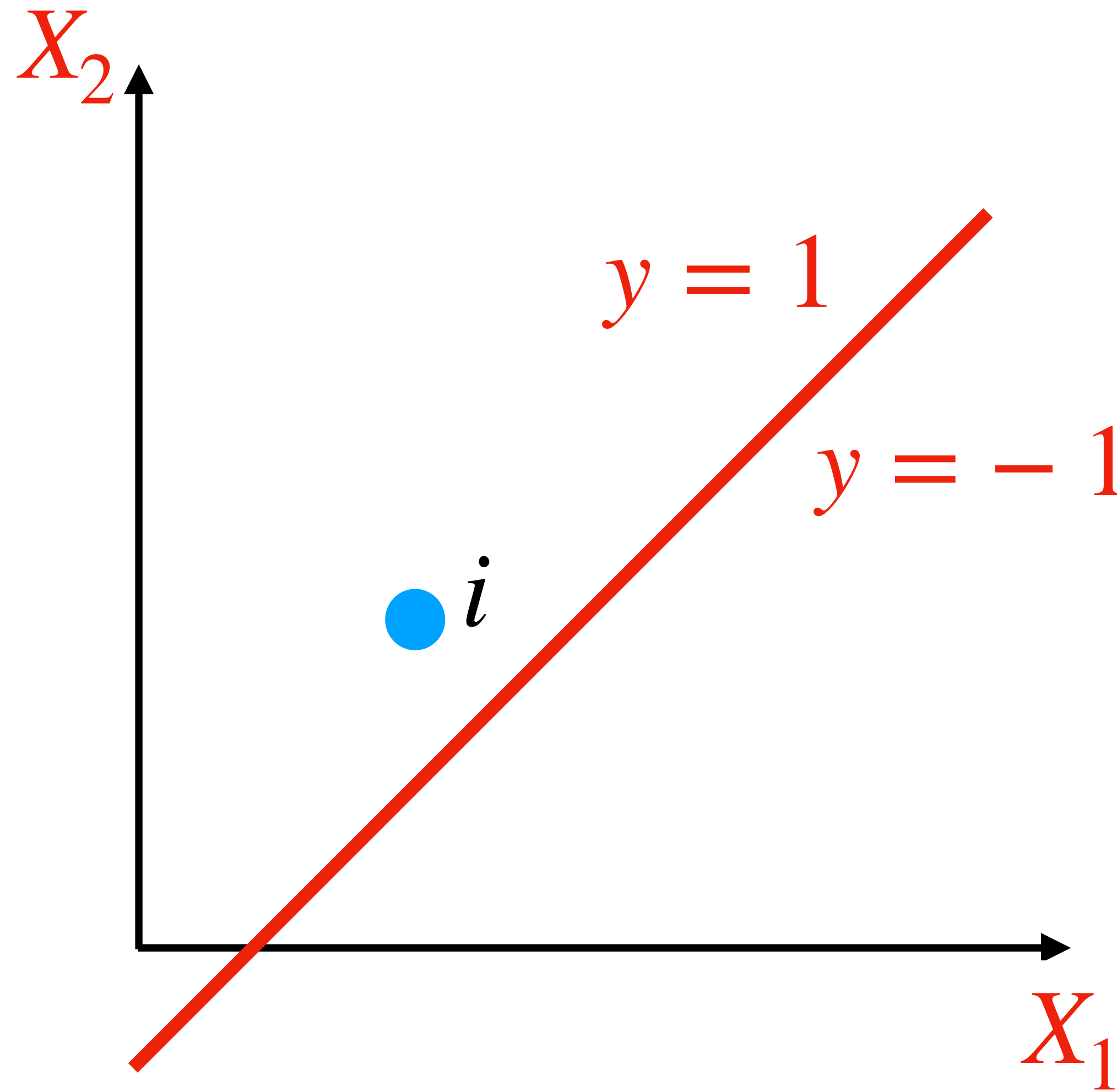
SUPPORT VECTOR MACHINES (SVM)

We want to find the hyperplane that 'separates' the classes 'well'.

Meaning of 'separates classes':

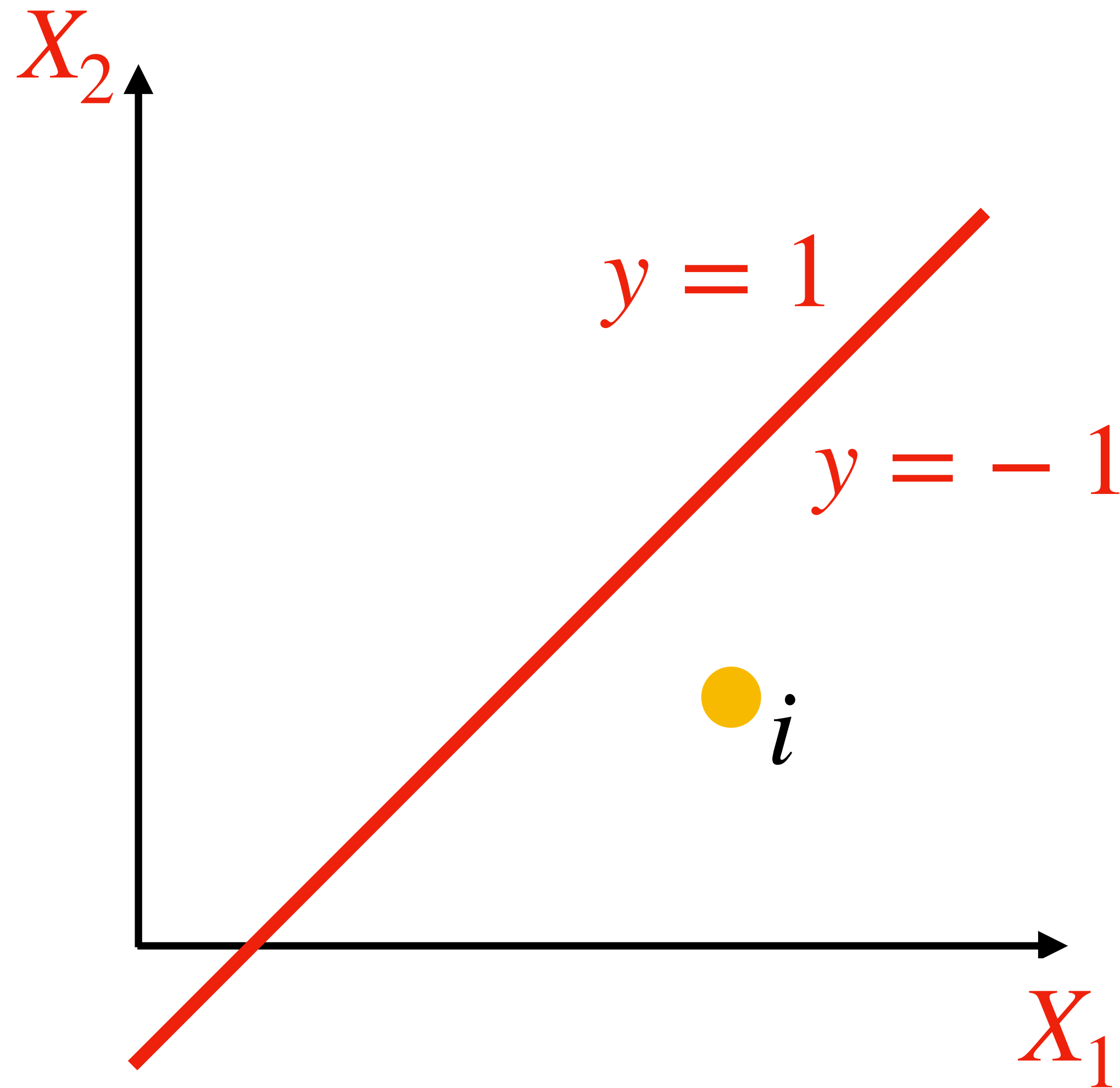
For a training point i with $y_i = 1$:

$$w_0 + w_1x_{i1} + w_2x_{i2} > 0$$



SUPPORT VECTOR MACHINES (SVM)

We want to find the hyperplane that 'separates' the classes 'well'.



Meaning of 'separates classes':

For a training point i with $y_i = 1$:

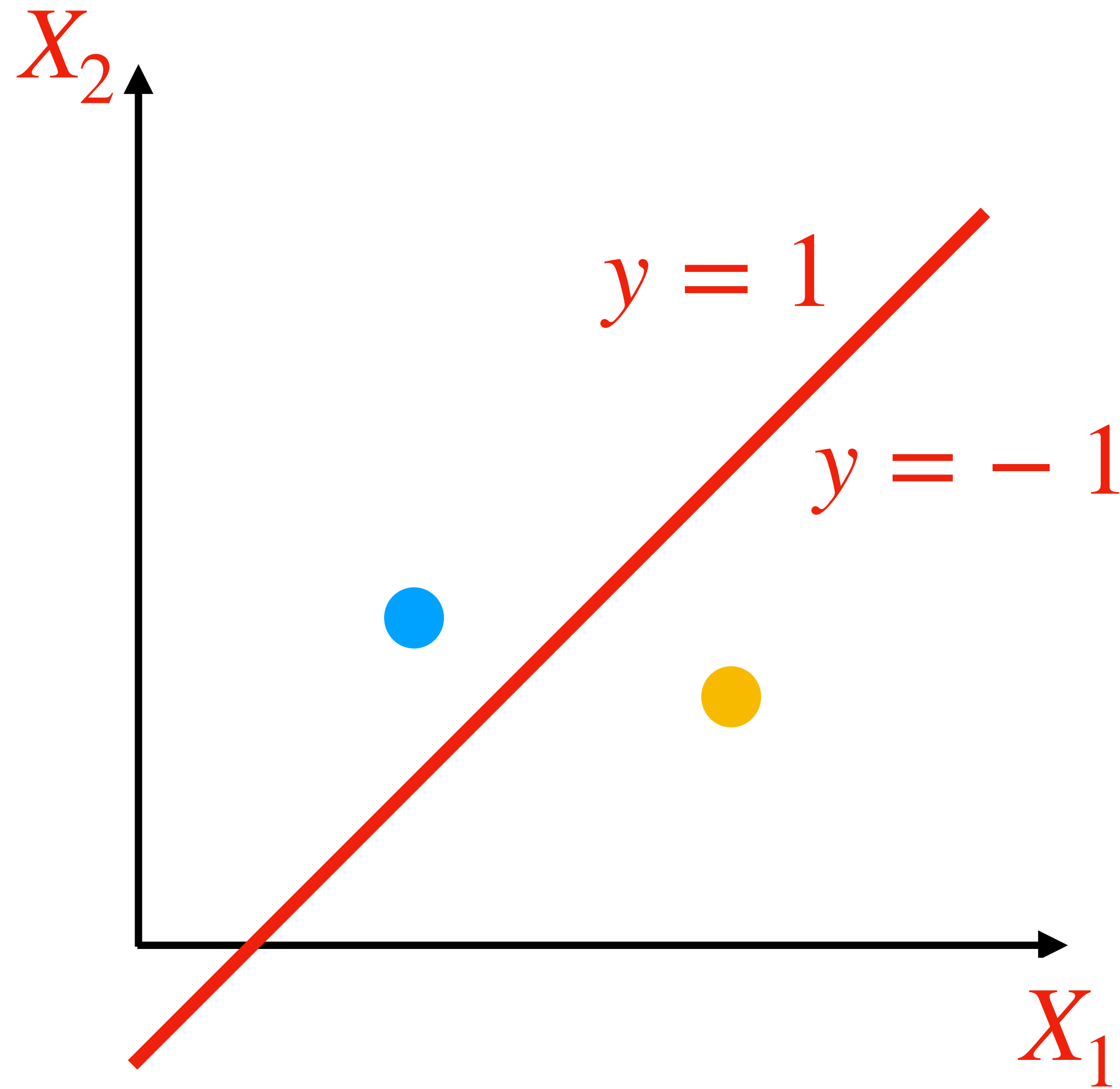
$$w_0 + w_1x_{i1} + w_2x_{i2} > 0$$

For a training point i with $y_i = -1$:

$$w_0 + w_1x_{i1} + w_2x_{i2} < 0$$

SUPPORT VECTOR MACHINES (SVM)

We want to find the hyperplane that 'separates' the classes 'well'.



Meaning of 'separates classes':

For a training point i with $y_i = 1$:

$$w_0 + w_1x_{i1} + w_2x_{i2} > 0$$

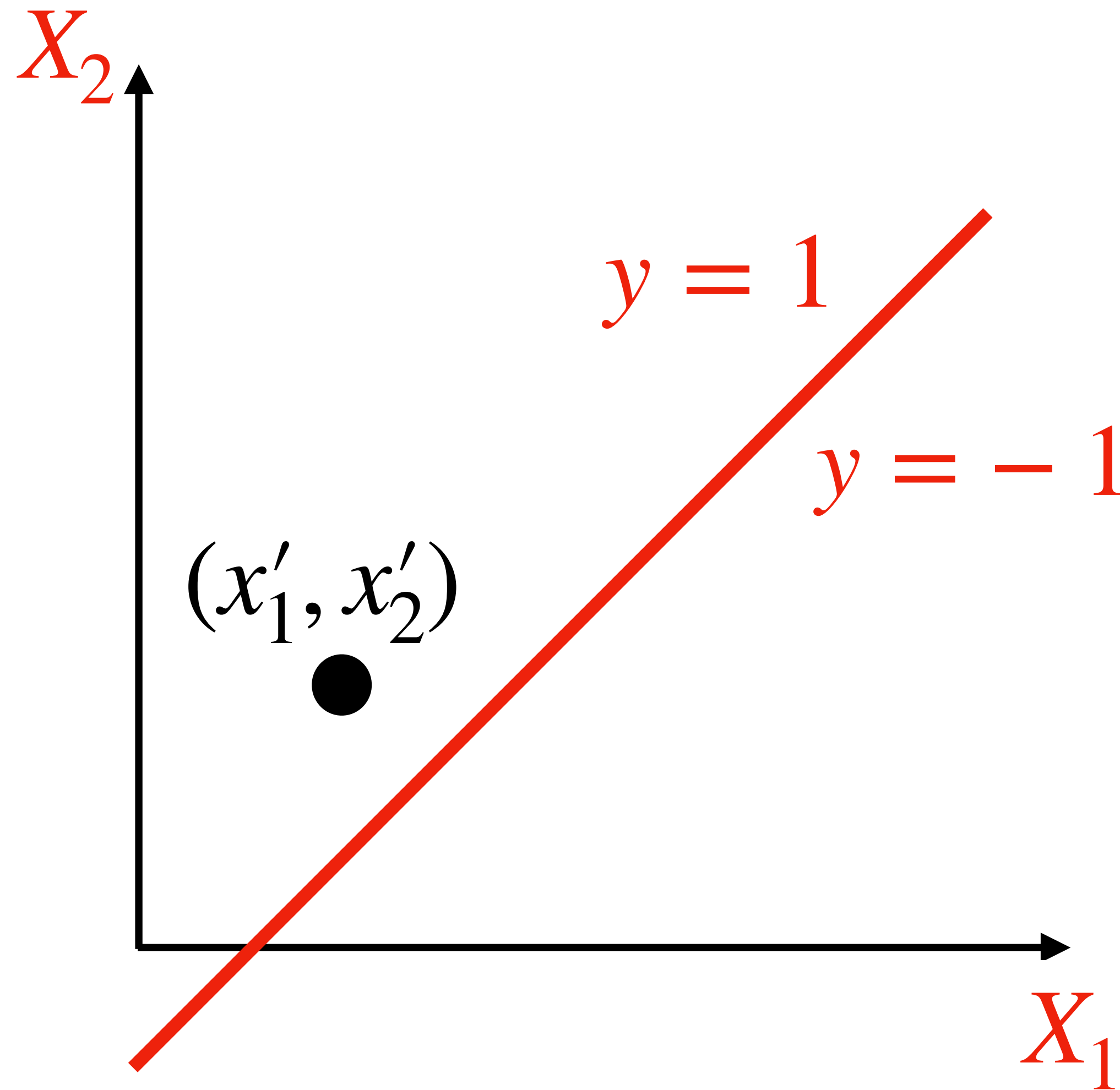
For a training point i with $y_i = -1$:

$$w_0 + w_1x_{i1} + w_2x_{i2} < 0$$

Can combine them in one inequality:

$$y_i(w_0 + w_1x_{i1} + w_2x_{i2}) > 0$$

SUPPORT VECTOR MACHINES (SVM)

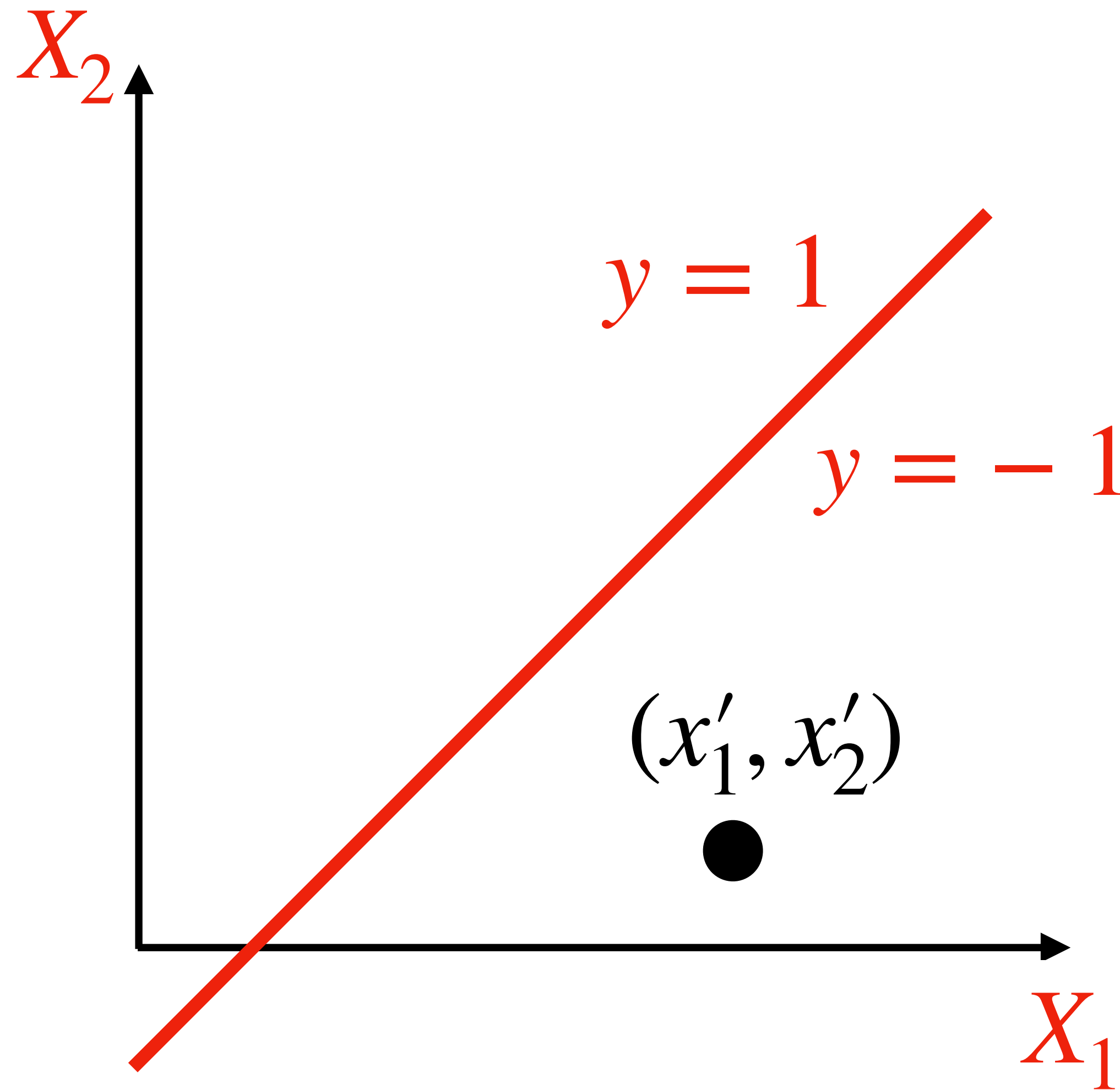


Given a test point (x'_1, x'_2) :

If $w_0 + w_1x'_1 + w_2x'_2 > 0$

Assign it to Class 1.

SUPPORT VECTOR MACHINES (SVM)



Given a test point (x'_1, x'_2) :

If $w_0 + w_1x'_1 + w_2x'_2 > 0$

Assign it to Class 1.

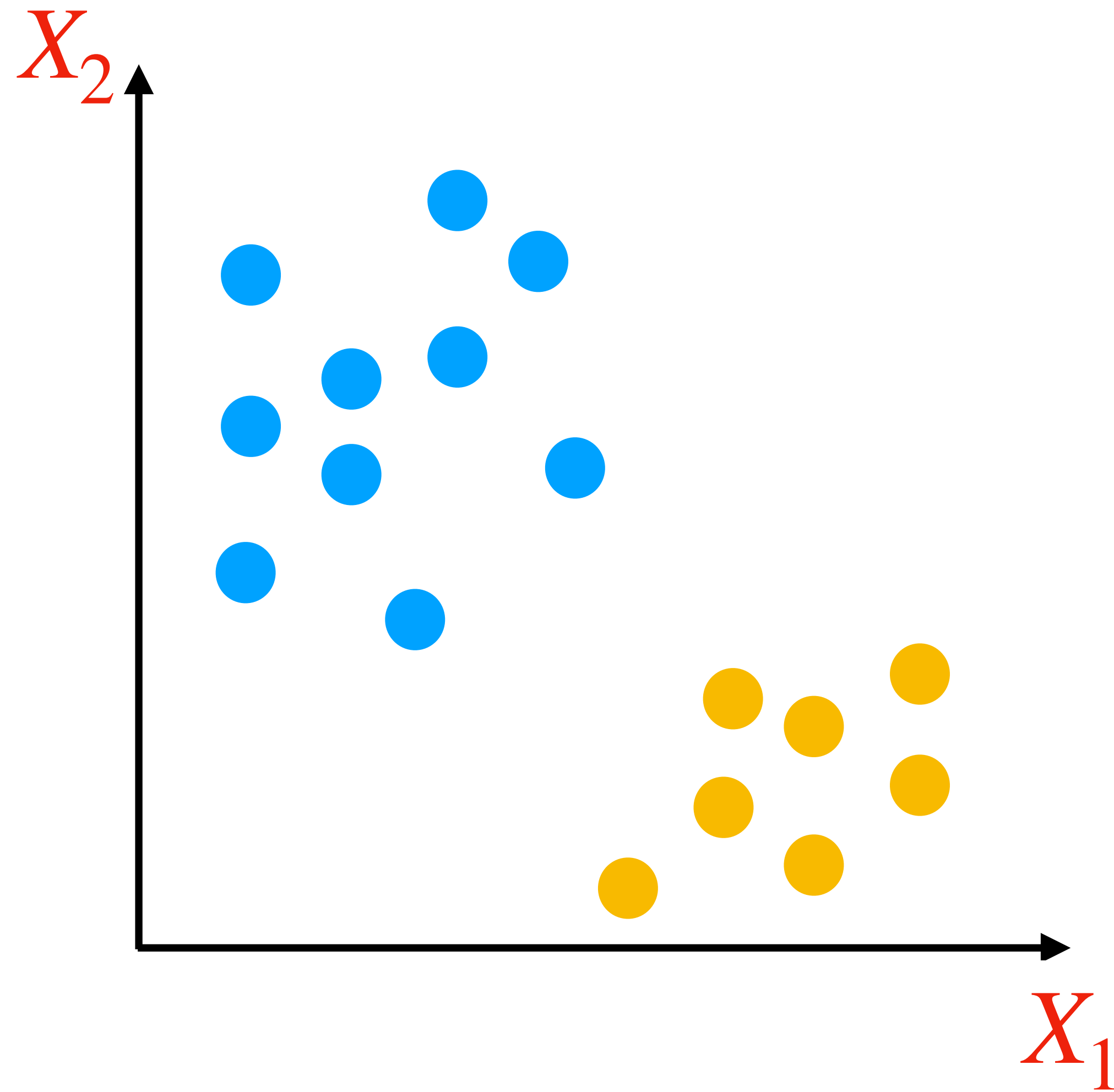
If $w_0 + w_1x'_1 + w_2x'_2 < 0$

Assign it to Class -1.

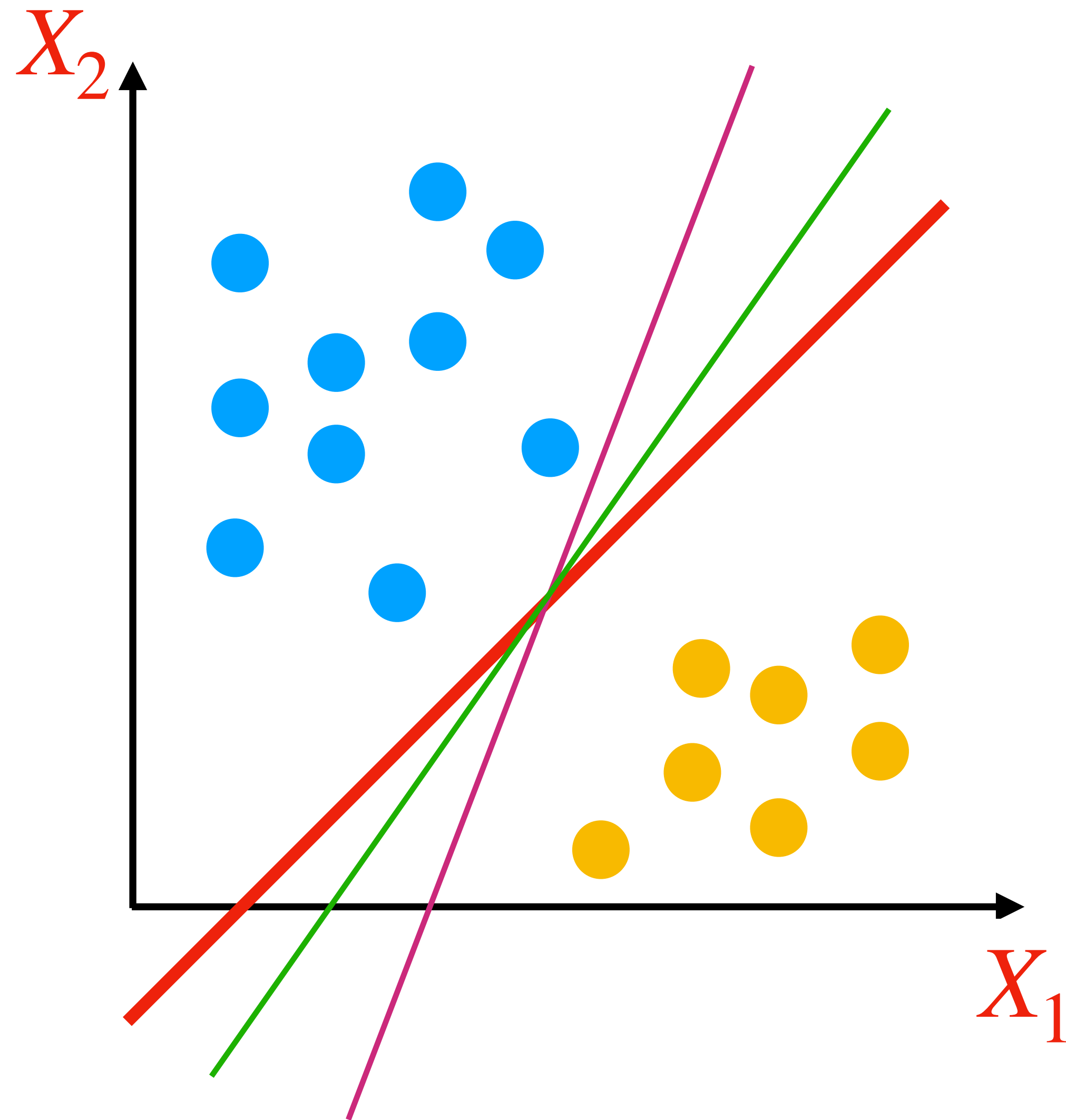
SUPPORT VECTOR MACHINES (SVM)

We want to find the hyperplane that
'separates' the classes 'well'.

How about 'separates classes well':



SUPPORT VECTOR MACHINES (SVM)

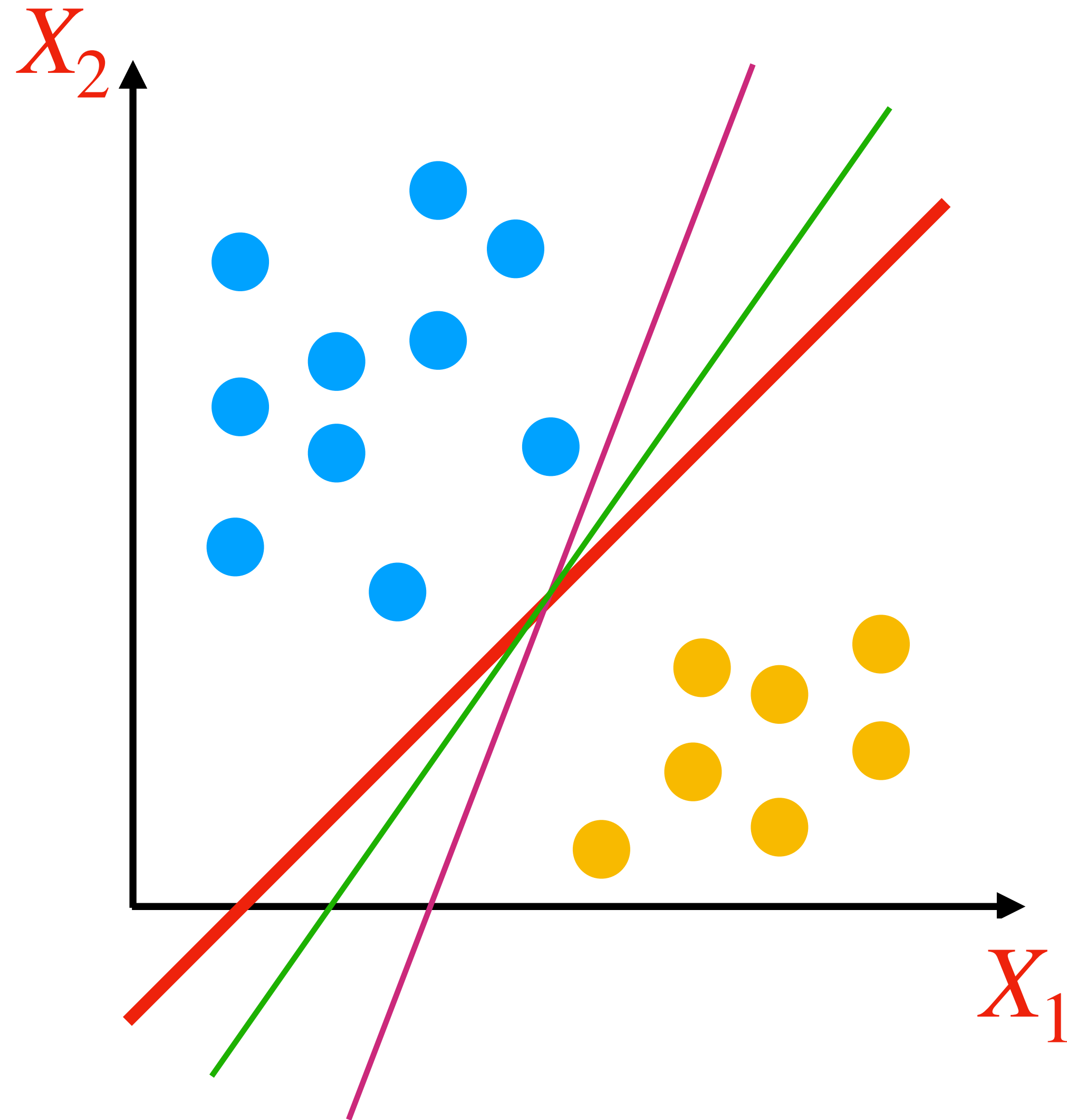


How about ‘separates classes well’:

Among red, green, purple:

red line separates the classes best.

SUPPORT VECTOR MACHINES (SVM)

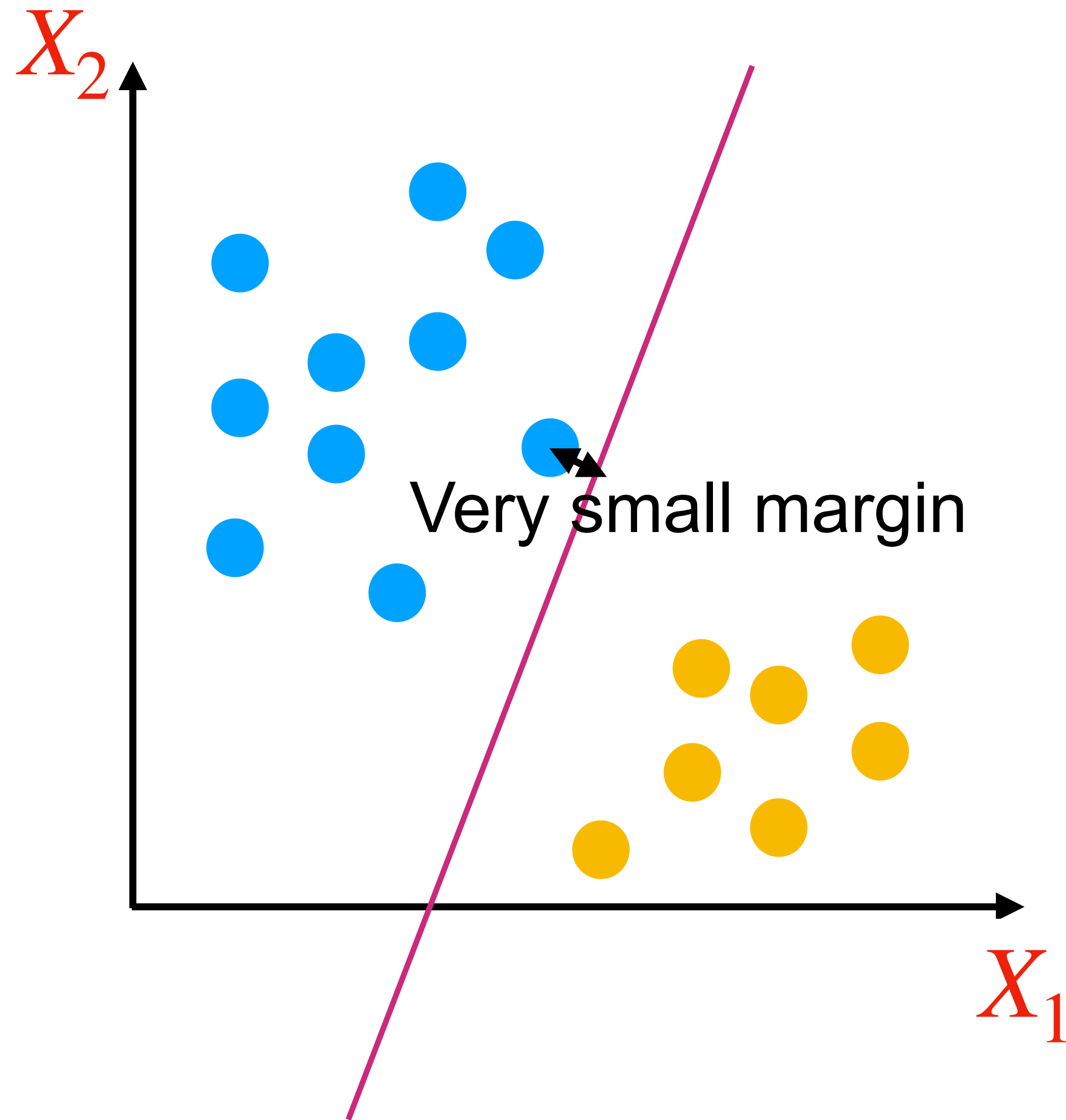


How about ‘separates classes well’:

Among red, green, purple:
red line separates the classes best.

Margin: minimum distance from any training point to the hyperplane.

SUPPORT VECTOR MACHINES (SVM)



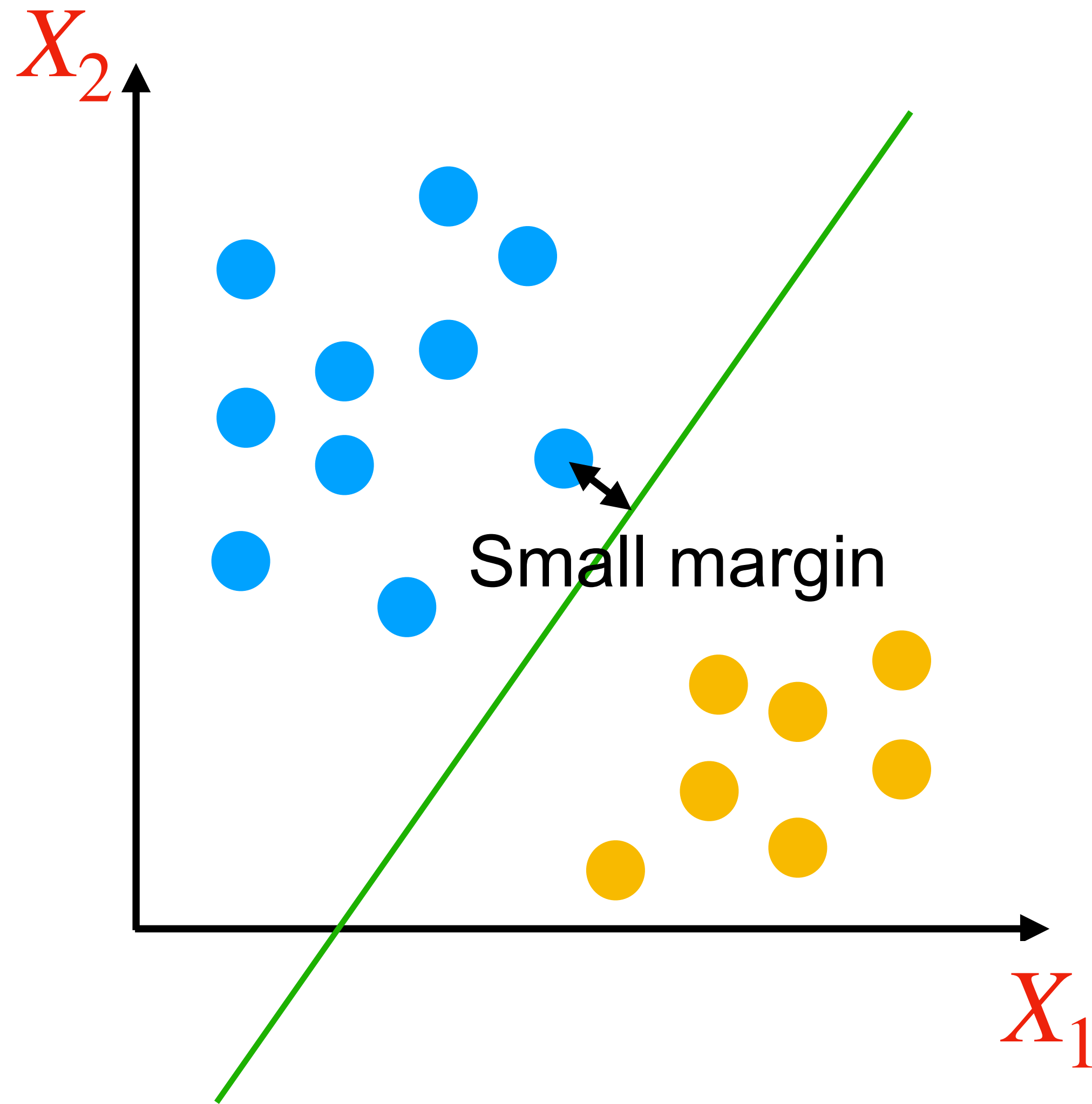
How about ‘separates classes well’:

Among red, green, purple:

red line separates the classes best.

Margin: minimum distance from any training point to the hyperplane.

SUPPORT VECTOR MACHINES (SVM)



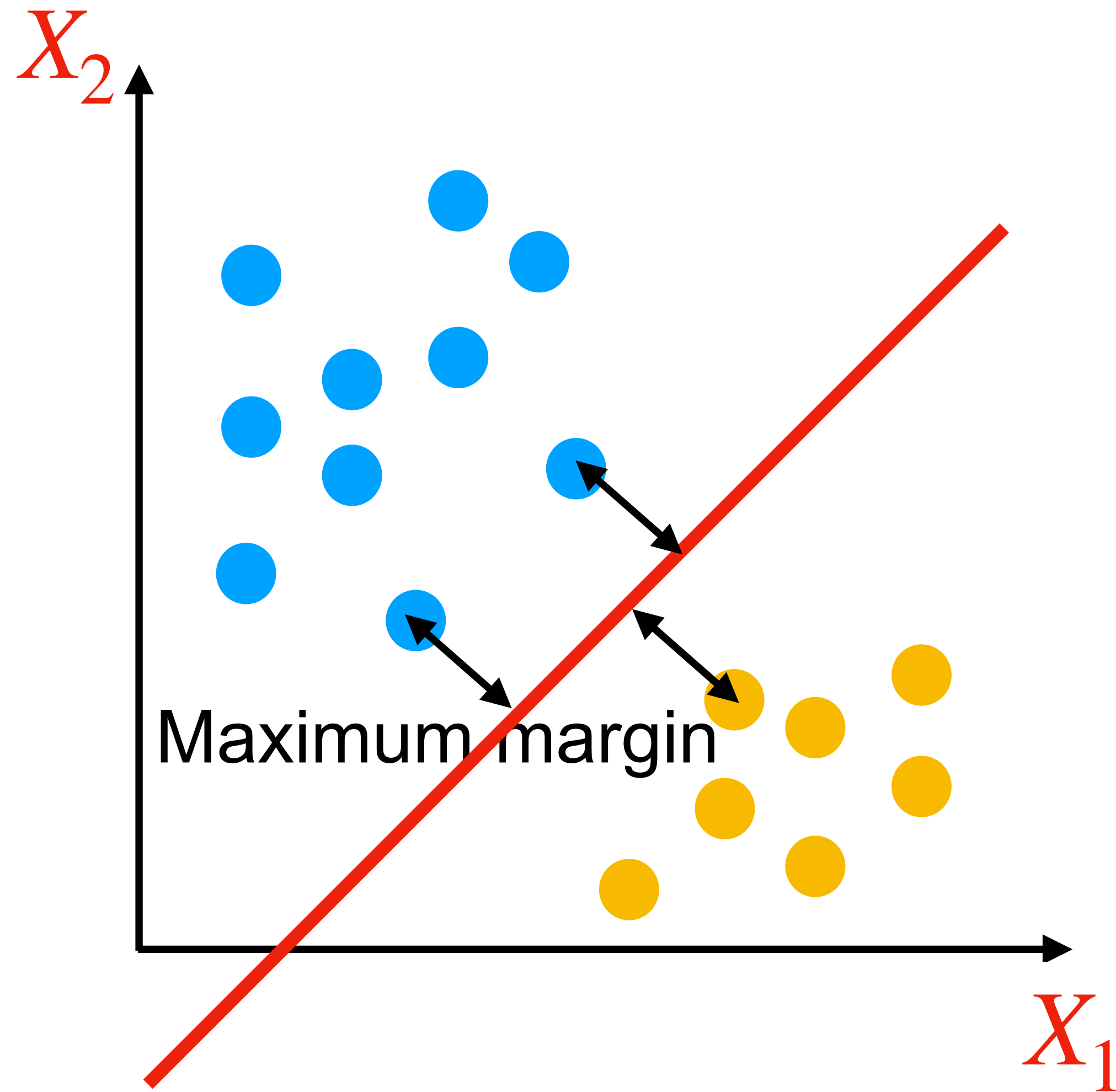
How about ‘separates classes well’:

Among red, green, purple:

red line separates the classes best.

Margin: minimum distance from any training point to the hyperplane.

SUPPORT VECTOR MACHINES (SVM)



How about ‘separates classes well’:

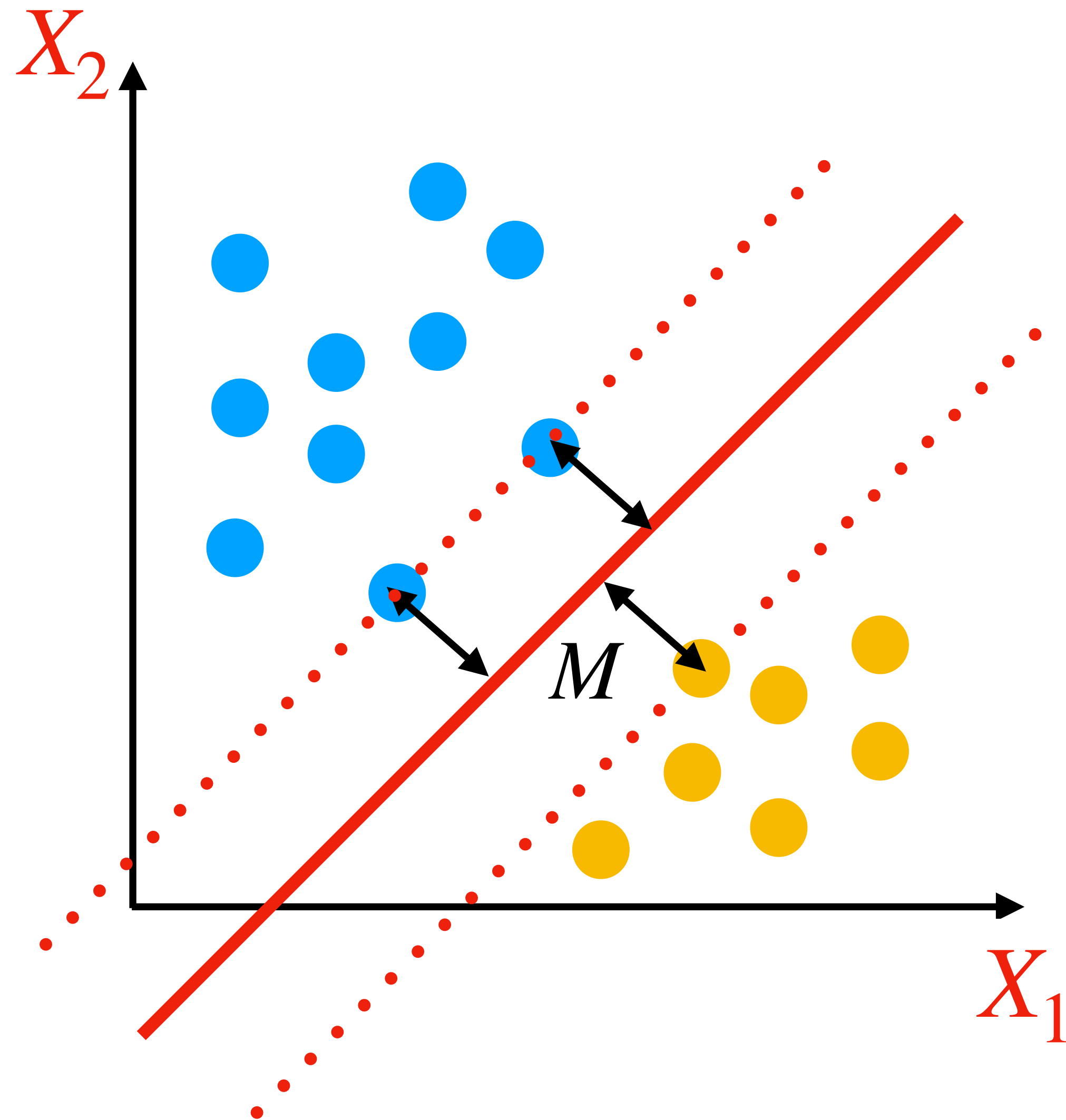
Among red, green, purple:

red line separates the classes best.

Margin: minimum distance from any training point to the hyperplane.

Red line has the maximum margin.

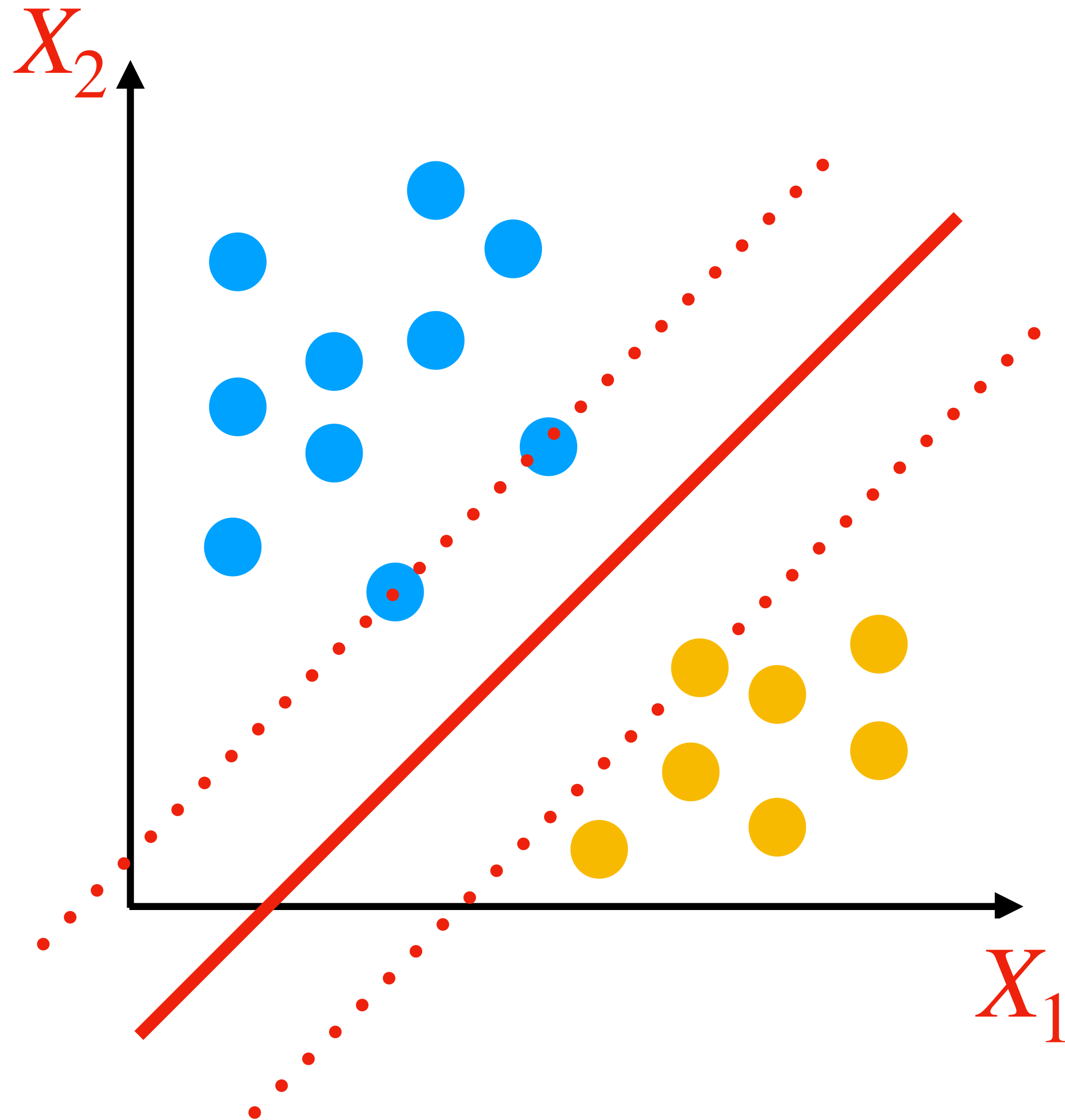
SUPPORT VECTOR MACHINES (SVM)



The points with maximum margin M distance from separating hyperplane are called the **support vectors**.

In this example we have 3 of them.

SUPPORT VECTOR MACHINES (SVM)



The points with maximum margin M distance from separating hyperplane are called the **support vectors**.

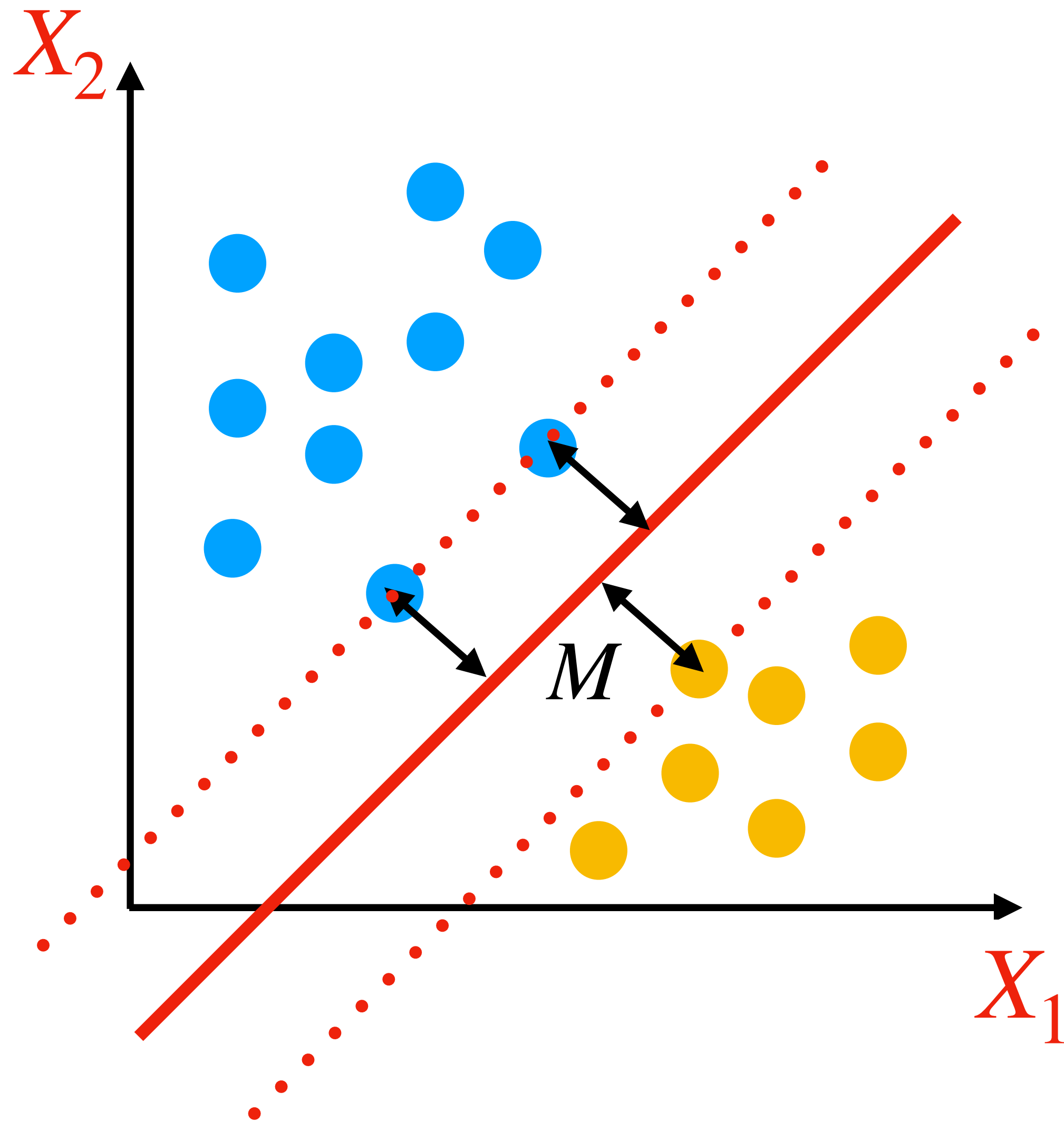
In this example we have 3 of them.

No point lies inside the margin.

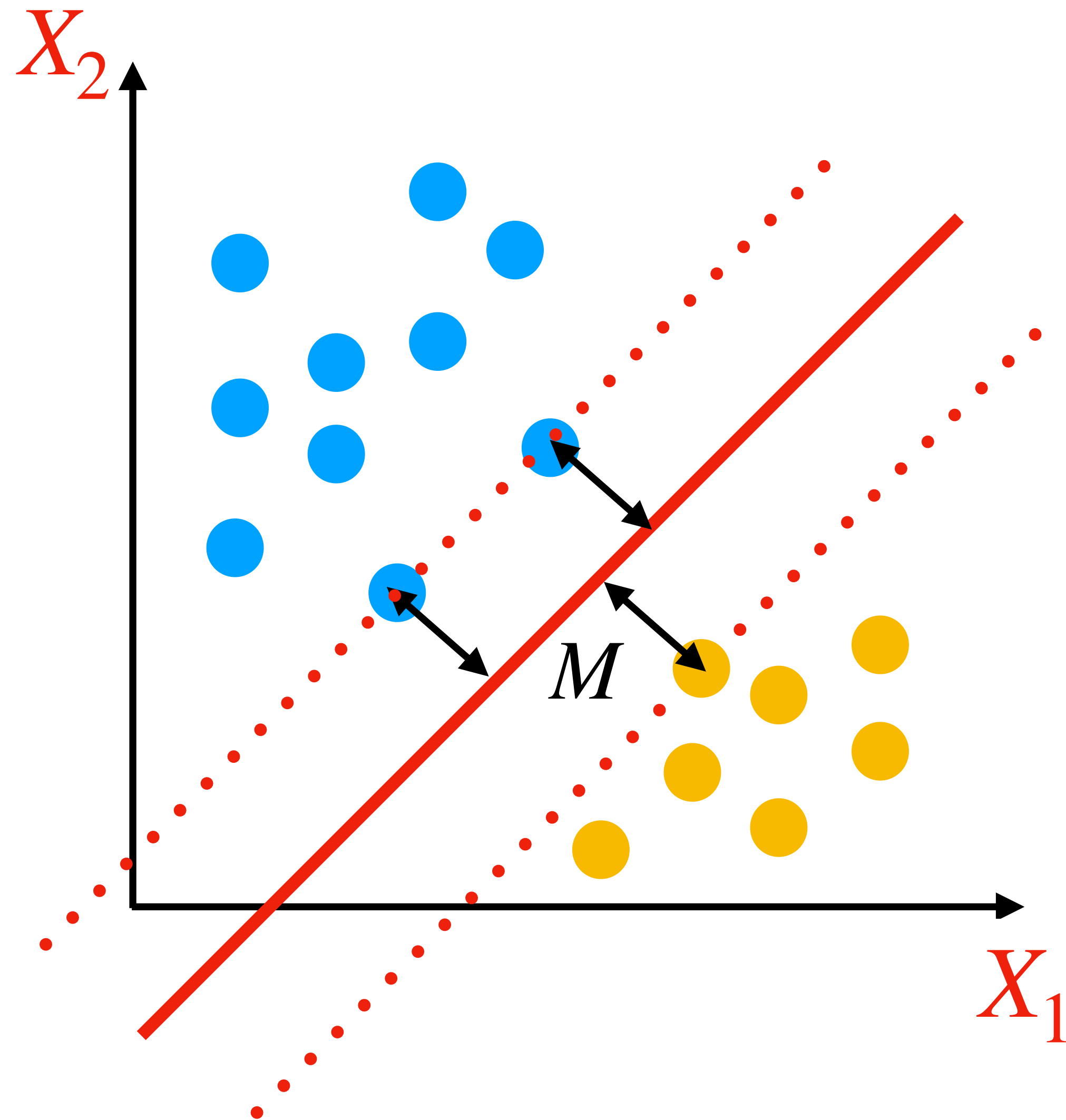
SUPPORT VECTOR MACHINES (SVM)

Informally the goal is to:

Find hyperplane that provides maximum margin M .



SUPPORT VECTOR MACHINES (SVM)



Formally the goal is to:

Find w_0, w_1, w_2 that

maximizes M

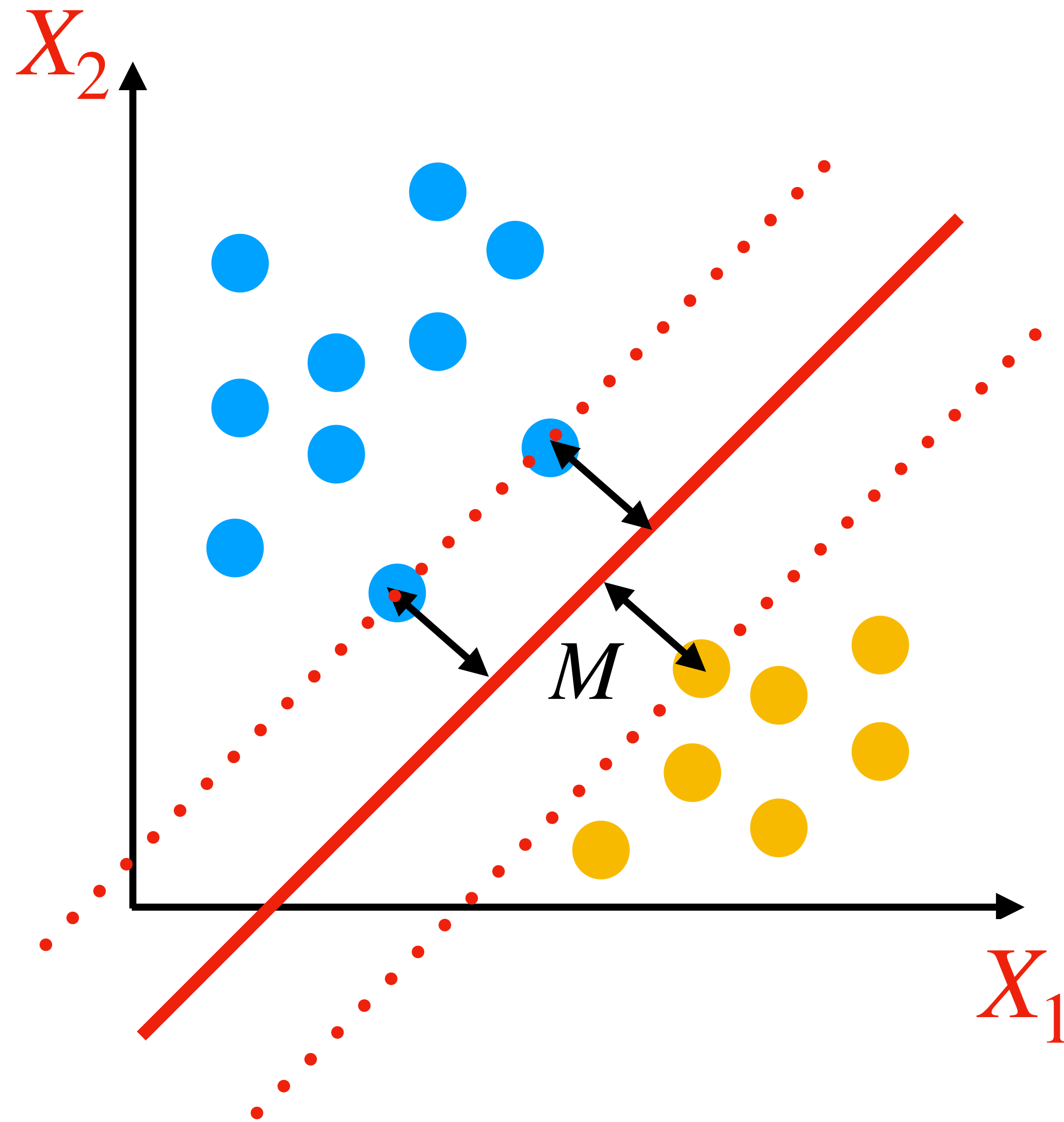
with the constraints

$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

$$\mathcal{C}_2 \quad y_i(w_0 + w_1x_{i1} + w_2x_{i2}) \geq M$$

for any training point i

SUPPORT VECTOR MACHINES (SVM)



Formally the goal is to:

Find w_0, w_1, w_2 that

maximizes M

with the constraints

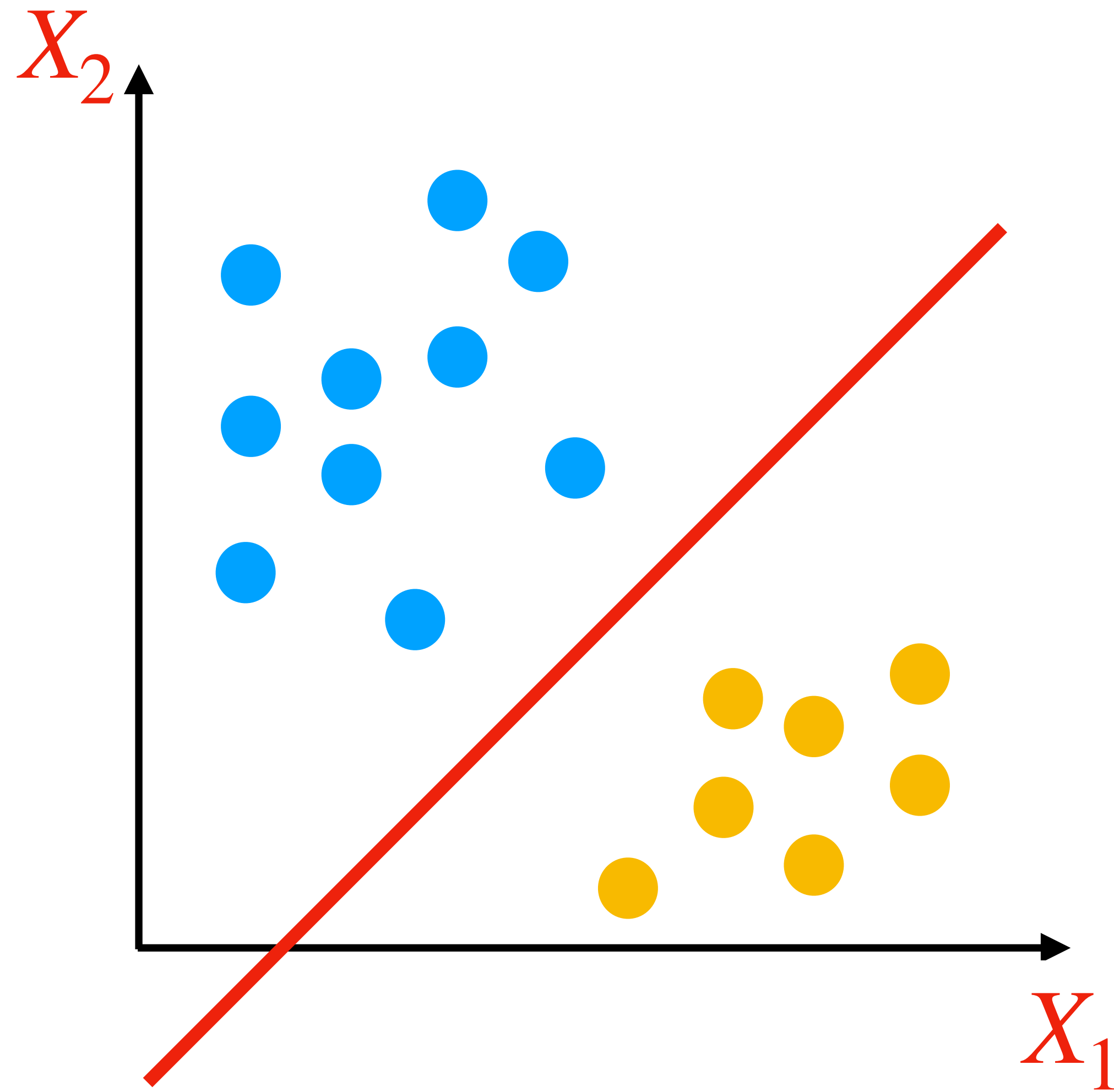
$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

$$\mathcal{C}_2 \quad y_i(w_0 + w_1x_{i1} + w_2x_{i2}) \geq M$$

for any training point i

Idea: Due to $\mathcal{C}_1$, $y_i(w_0 + w_1x_{i1} + w_2x_{i2})$ is distance of point i to hyperplane.

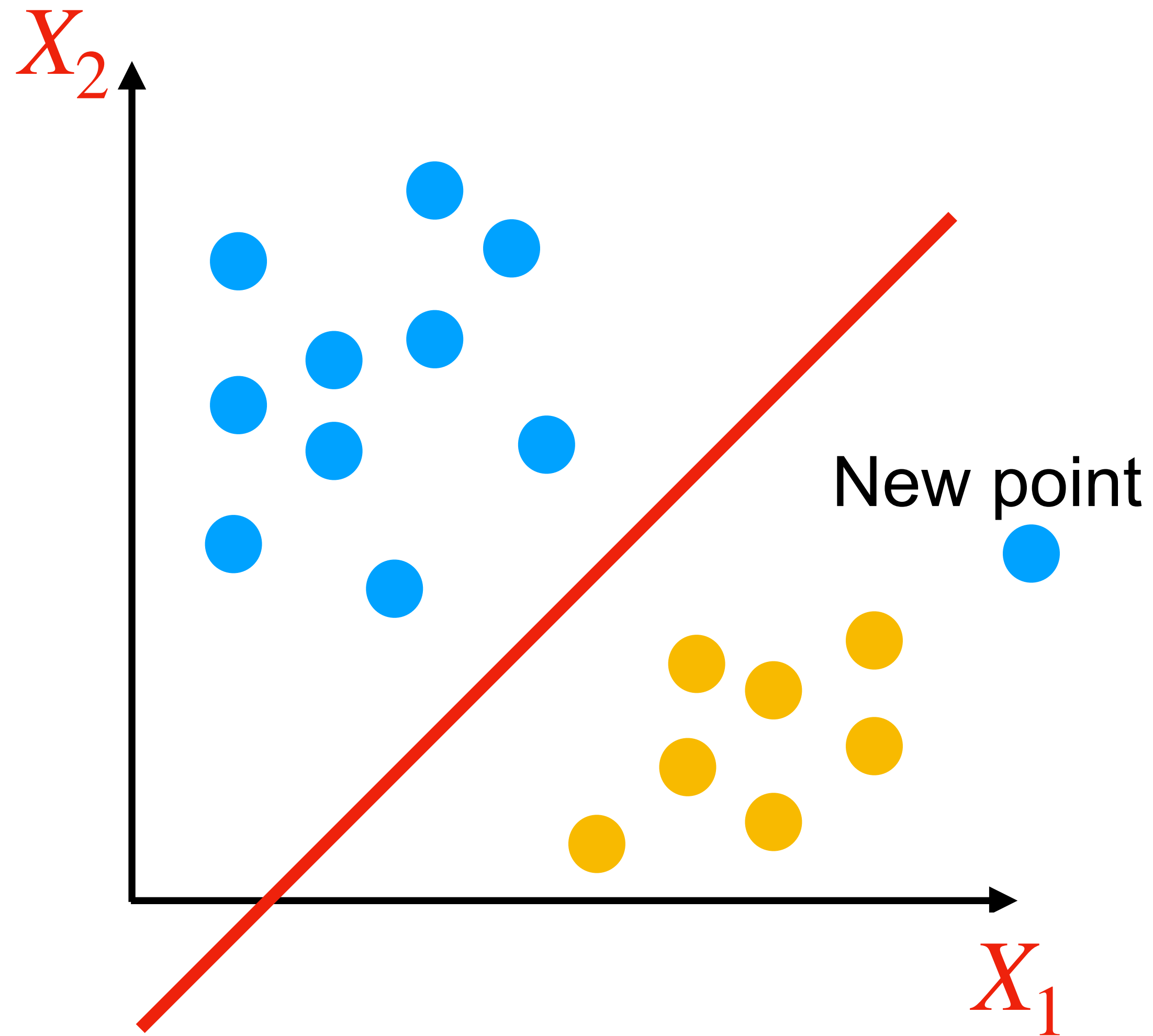
SUPPORT VECTOR MACHINES (SVM)



Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.

SUPPORT VECTOR MACHINES (SVM)



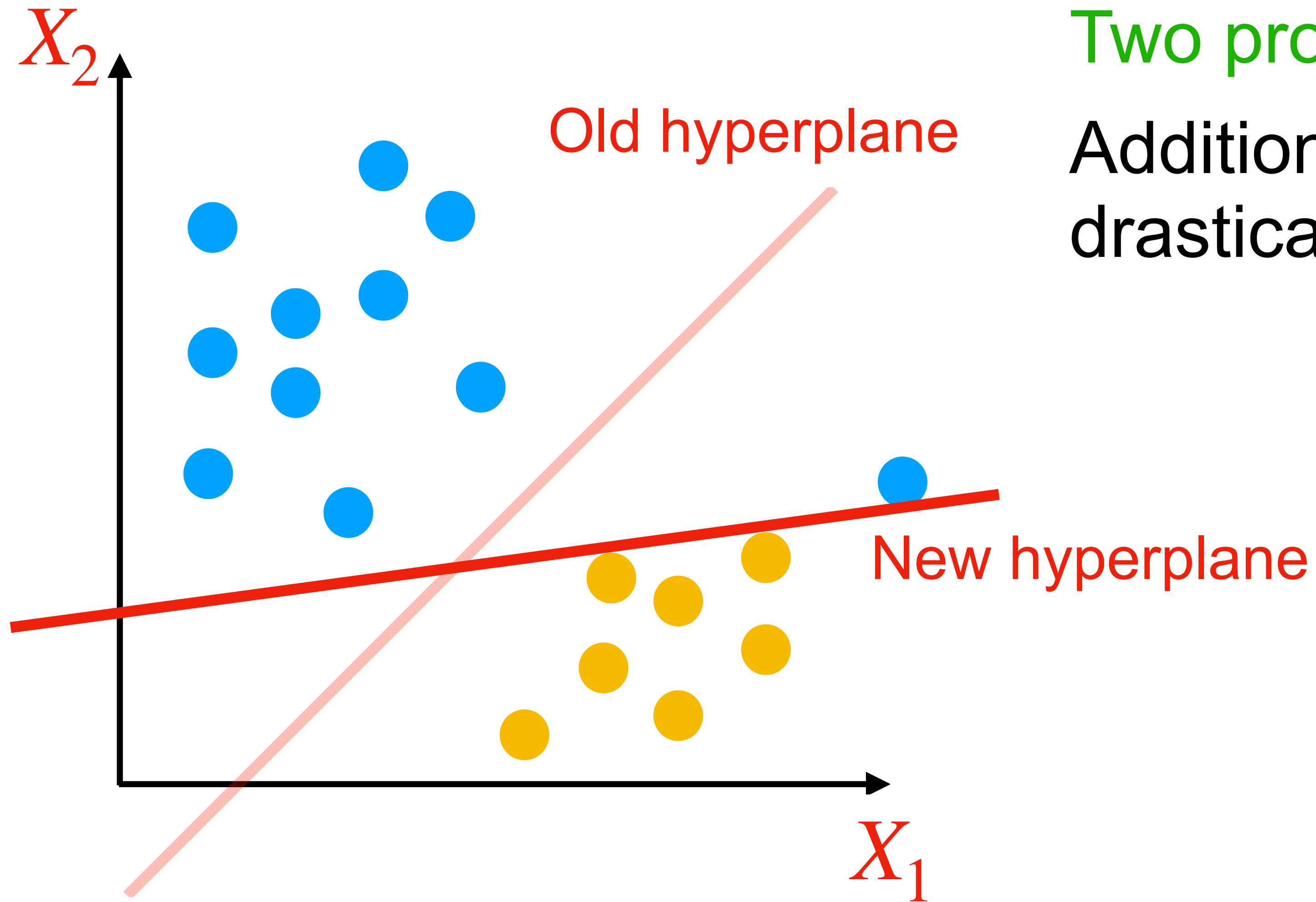
Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.

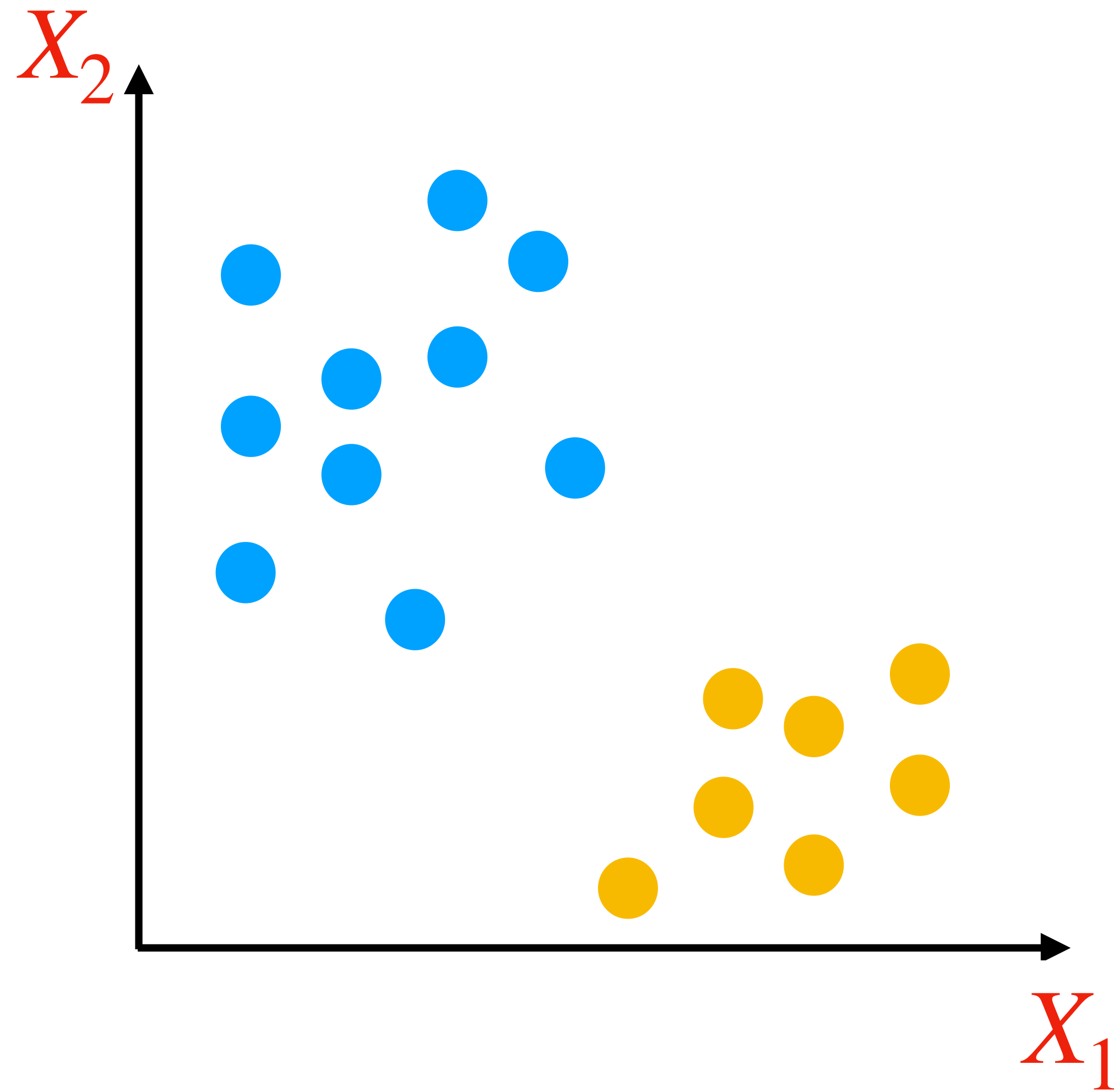
SUPPORT VECTOR MACHINES (SVM)

Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.



SUPPORT VECTOR MACHINES (SVM)



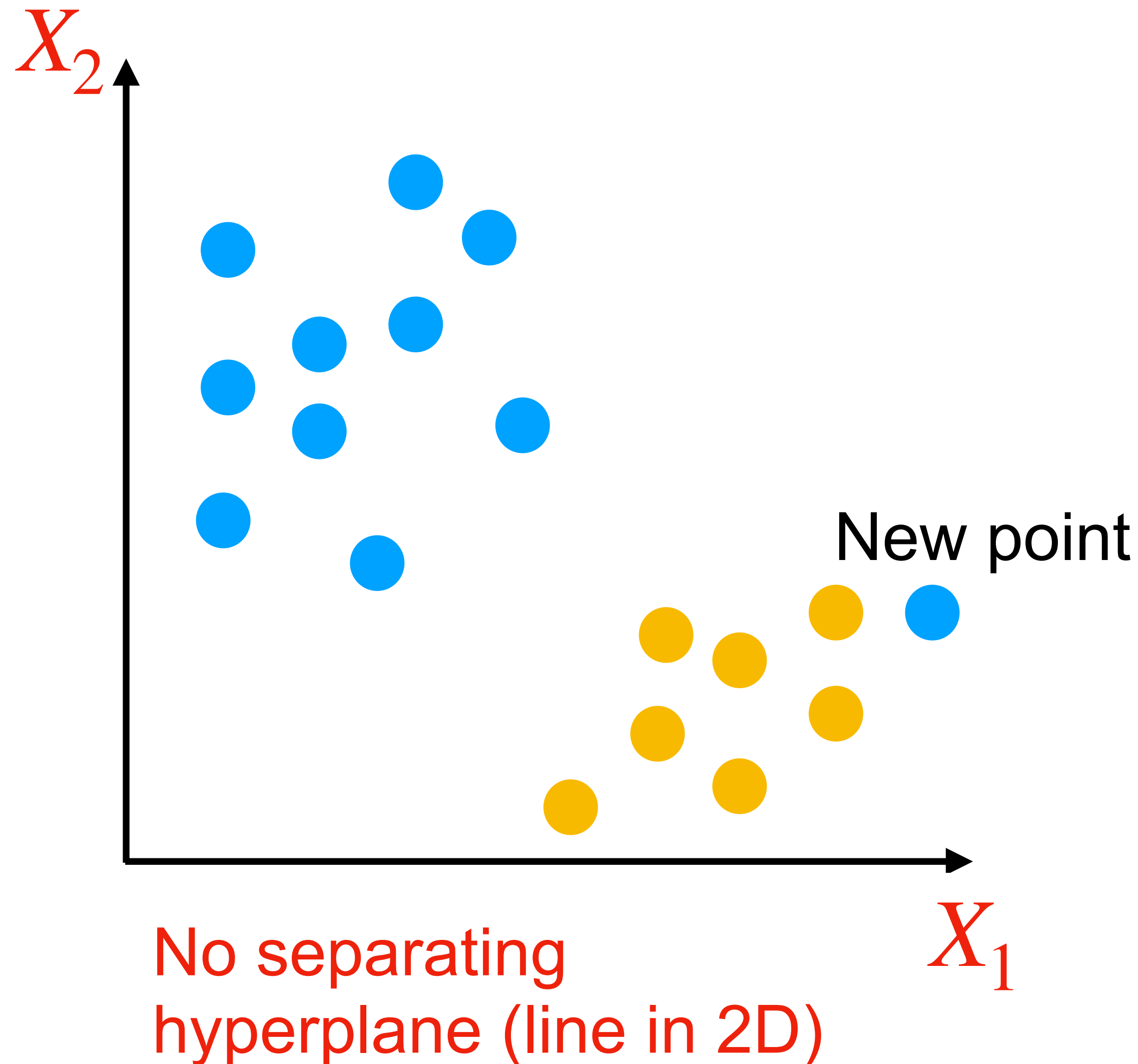
Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.

Insisting on perfect separation may cause overfitting!

It may not even be possible to find a separating hyperplane (line in 2D).

SUPPORT VECTOR MACHINES (SVM)



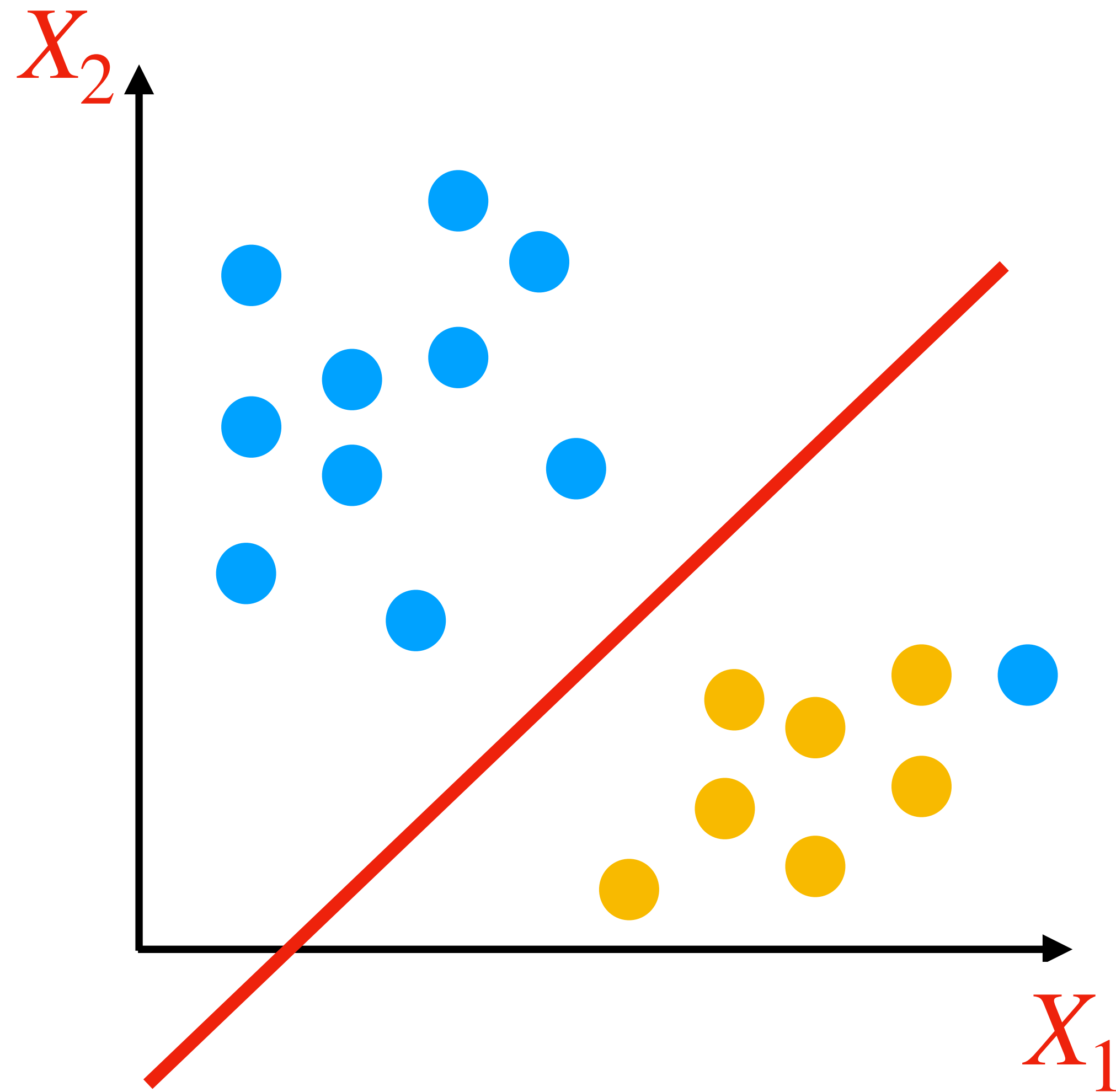
Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.

Insisting on perfect separation may cause overfitting!

It may not even be possible to find a separating hyperplane (line in 2D).

SUPPORT VECTOR MACHINES (SVM)



Two problems with the same solution:

Addition of a single point might drastically change the hyperplane.

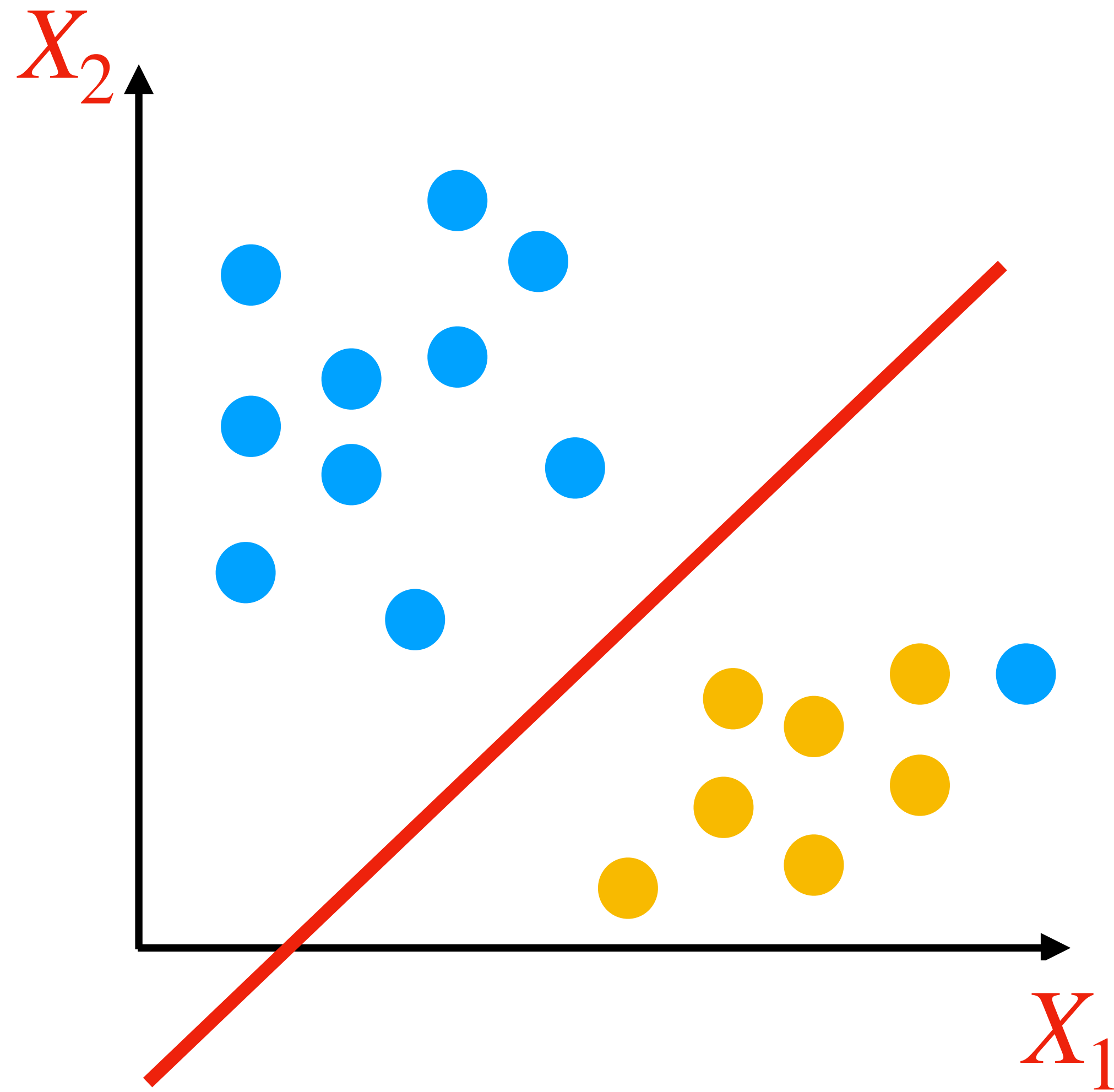
Insisting on perfect separation may cause overfitting!

It may not even be possible to find a separating hyperplane (line in 2D).

Solution is to use **Soft Margin**:

Add flexibility; some observations can be on the wrong side.

SUPPORT VECTOR MACHINES (SVM)



Soft Margin:

Find w_0, w_1, w_2 that
maximizes M
with the constraints

$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

$$\mathcal{C}_2 \quad y_i(w_0 + w_1x_{i1} + w_2x_{i2}) \geq M(1 - \epsilon_i) \\ \text{for any training point } i$$

$$\mathcal{C}_3 \quad \epsilon_i \geq 0, \sum_{i=1}^n \epsilon_i \leq C$$

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?

As before due to $\mathcal{C}_1$
 $y_i(w_0 + w_1x_{i1} + w_2x_{i2})$ is
distance of point i to hyperplane

$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?

Don't insist distance be at least M for all training points.

Each training point i has some slack ϵ_i .

$\epsilon_i > 0$ allows point i to be inside the margin.

$\epsilon_i > 1$ allows point i to be on the wrong side of the hyperplane.

$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

$\mathcal{C}_2$

$$y_i(w_0 + w_1x_{i1} + w_2x_{i2}) \geq M(1 - \epsilon_i)$$

for any training point i

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?

C : total budget for all the slacks.
(hyperparameter tuned via cross-validation)

Controls bias-variance tradeoff:

Small $C \Rightarrow$ narrow margins that should not be violated (low bias, high variance).

Large $C \Rightarrow$ wide margins, points allowed on the wrong side of hyperplane (high bias, low variance).

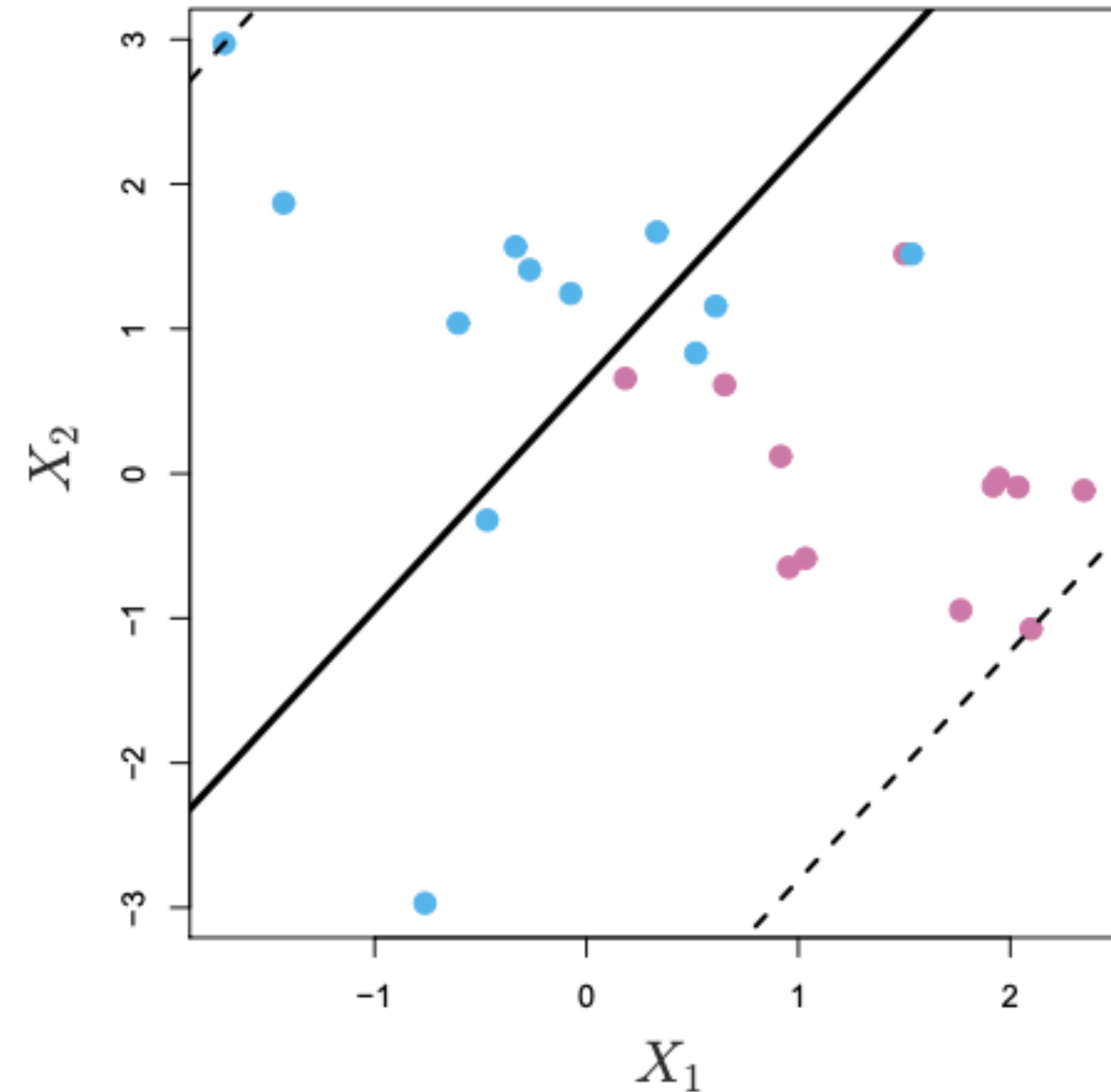
$$\mathcal{C}_1 \quad w_1^2 + w_2^2 = 1$$

$$\mathcal{C}_2 \quad y_i(w_0 + w_1x_{i1} + w_2x_{i2}) \geq M(1 - \epsilon_i) \\ \text{for any training point } i$$

$$\mathcal{C}_3 \quad \epsilon_i \geq 0, \sum_{i=1}^n \epsilon_i \leq C$$

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?

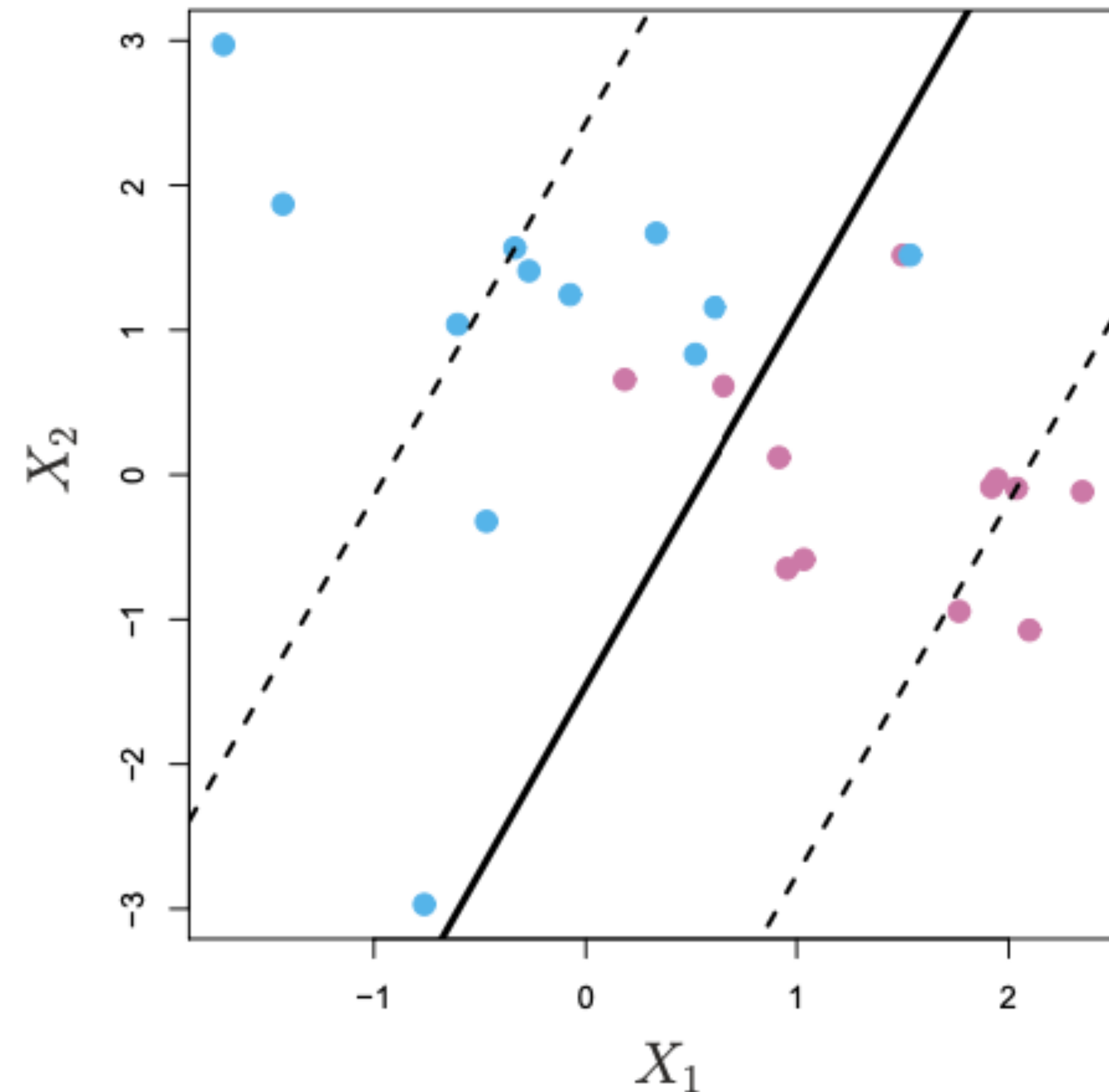


Very large C .

(Many points inside margin, many points on the wrong side of the line)

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?

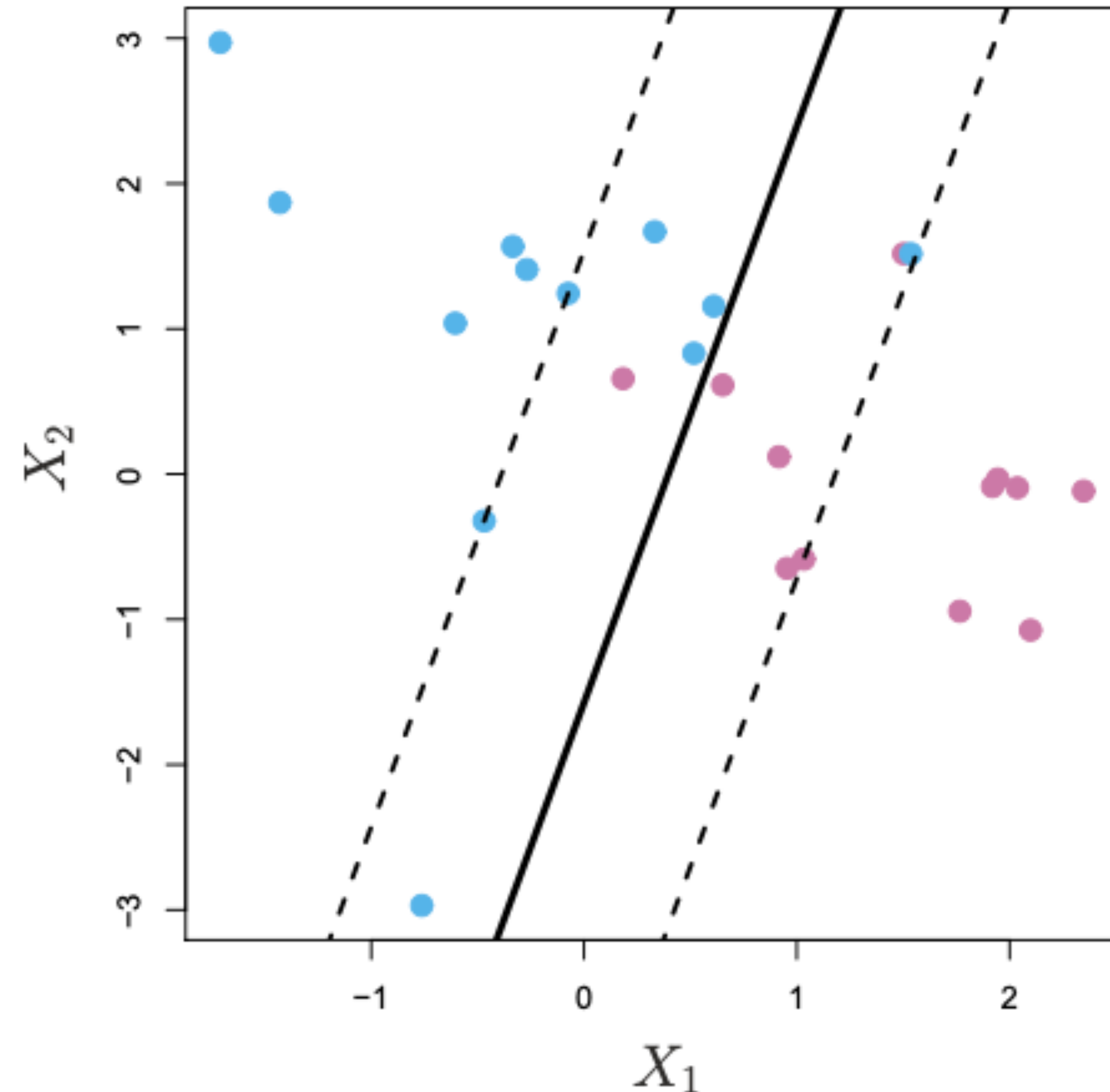


Smaller C .

(Fewer points inside margin, fewer points on the wrong side of the line)

SUPPORT VECTOR MACHINES (SVM)

What does all this mean?



Even Smaller C .

(Even fewer points inside margin, very few points on the wrong side of the line)